



**UNIVERSITÉ
DE GENÈVE**



**UNIVERSITÉ
DE GENÈVE**

FACULTÉ DES SCIENCES

UNIVERSITÉ DE GENÈVE

FACULTÉ DES SCIENCES

Département d'Informatique

Travail de Bachelor en Sciences Informatiques

Année académique 2025-2026

Programmation dynamique pour formules SAT

Auteur :

Elie BUSSOD

Superviseurs :

Prof. Arnaud CASTEIGTS

Prof. Pierre LEONE

Date de rendu : Juin 2026

Résumé

Nous implémentons en C le comptage de modèles ($\#SAT$) et la maximisation de clauses (MaxSAT) par programmation dynamique sur branch decomposition, dont la complexité est paramétrée par la ps-width (algorithme de Saether, Telle et Vatshelle). L'implémentation est étendue de manière à produire, au cours de la même exécution, un DAG d-DNNF structuré (construction de Bova, Capelli, Mengel et Slivovsky), sur lequel six requêtes polytimes sont évaluées. L'évaluation empirique montre qu'un solveur de production reste plus rapide en résolution unique, mais que le DAG compilé s'amortit dès qu'une même formule est interrogée de nombreuses fois, ce qui place la contribution du côté de la compilation de connaissances plutôt que de la résolution.

Table des matières

Résumé	1
1 Introduction	5
1.1 Contexte et motivation	5
1.2 Approche structurale et décomposition arborescente	6
1.3 Objectifs de ce travail	6
2 État de l’art	7
2.1 SAT, #SAT et MaxSAT	7
2.1.1 Le problème SAT	7
2.1.2 #SAT : comptage de modèles	7
2.1.3 MaxSAT	7
2.2 Algorithmes de résolution	8
2.2.1 DPLL et CDCL	8
2.2.2 Extensions pour #SAT et MaxSAT	8
2.2.3 Compilation de connaissances	8
2.3 Décompositions arborescentes et paramètres de largeur	8
2.4 ps-width et l’algorithme de Saether-Telle-Vatshelle	9
2.5 Compilation de connaissances CNF $\rightarrow$ d-DNNF	9
2.5.1 Langages cibles	9
2.5.2 Requêtes tractables sur d-DNNF	10
2.5.3 Compilation explicite pilotée par la ps-width : BCMS 2015	10
3 Cadre théorique	11
3.1 Notations et préliminaires	11
3.2 Branch decomposition et ps-width	11
3.2.1 Branch decomposition d’une formule	11
3.2.2 Coupes et PS-sets	12
3.2.3 ps-value et ps-width	13
3.3 Programmation dynamique pour #SAT et MaxSAT	13
3.3.1 Stratégie d’ensemble	13
3.3.2 Procédure 1 : calcul bottom-up des $PS'(F_v)$	14
3.3.3 Procédure 2 : calcul top-down des $PS'(F_{\bar{v}})$	15
3.3.4 Procédure 3 : table DP $Tab_v(C, C')$	15
3.3.5 Théorèmes 2 et 3	16
3.4 Heuristique GreedyOrder	17
3.5 Compilation de F en d-DNNF structuré (BCMS)	18
3.5.1 Construction du DAG par récurrence	18
3.5.2 Propriétés du DAG produit	19
3.5.3 Borne de taille (Lemme 7 et Théorème 1)	19
3.6 Requêtes polytimes sur d-DNNF	20
3.6.1 Trois propriétés clés	20

3.6.2	CT : model counting	20
3.6.3	CO, VA et ME	21
3.6.4	Transformations internes : conditionnement et lissage	21
3.6.5	CE : clausal entailment	21
3.6.6	IM : implicant check	22
4	Implémentation	23
4.1	Representation des PS-sets par bitsets 64-bit	23
4.2	Le Binary Trie : deduplication globale des PS-sets	23
4.3	Procédures 1 et 2 : produits cartesiens et masques de coupe	24
4.4	Procédure 3 : la boucle critique	24
4.5	Construction du DAG d-DNNF en parallèle du DP	24
4.6	Heuristique GreedyOrder et modes alternatifs	25
4.7	Requêtes polytimes sur d-DNNF : choix d'implémentation	25
4.8	Robustesse : gestion mémoire et erreurs	25
5	Résultats expérimentaux	27
5.1	Instances testées	27
5.2	Méthodologie expérimentale	29
5.2.1	Machine et environnement	29
5.2.2	Protocole de mesure en trois passes	29
5.2.3	Validation par invariants théoriques	30
5.2.4	Cas dégénérés	30
5.3	Résultats : programmation dynamique et compilation	31
5.3.1	ps-width observée et bornes théoriques	31
5.3.2	Temps DP par famille et comparaison à Z3	31
5.3.3	Impact du mode de construction d'arbre	32
5.3.4	Corrélation entre ps-width DP et conflits Z3	33
5.4	Résultats : requêtes sur DAG compilé	34
5.4.1	Coût par requête	34
5.4.2	Comparaison CO vs Z3 SAT sur la même question	35
5.4.3	Break-even N	35
6	Discussion	37
6.1	Du solveur au compilateur de connaissances	37
6.2	ps-width et effort CDCL : des difficultés orthogonales	37
6.3	GreedyOrder, maillon faible et principal levier	38
6.4	Limites de l'évaluation et améliorations possibles	38
7	Conclusion	40
7.1	Bilan et réponse à la question de recherche	40
7.2	Perspectives	40
	Déclaration d'utilisation de l'IA	44

Table des figures

3.1	Une branch decomposition (T, δ) d'une formule F sur cinq variables et quatre clauses, reprise de la Figure 2 de [18]. La formule a pour variables $\text{var}(F) = \{x_1, x_2, x_3, x_4, x_5\}$ et pour clauses $c_1 = \{x_1, x_2\}$, $c_2 = \{x_1, \neg x_2, x_3\}$, $c_3 = \{x_2, \neg x_4, x_5\}$, $c_4 = \{x_2, x_3, x_5\}$. Le noeud v marqué définit la coupe $\delta(v) = \{x_1, x_2, c_1, c_3\}$, soit $\mathcal{C} = \{c_1, c_3\}$ et $X = \{x_1, x_2\}$. Les deux sous-formules croisées sont $F_v = \{\{x_1, \neg x_2\}, \{x_2\}\}$ (versions de c_2 et c_4 restreintes aux variables de X) et $F_{\bar{v}} = \{\emptyset, \{\neg x_4, x_5\}\}$ (versions de c_1 et c_3 restreintes aux variables de $\bar{X}$).	12
5.1	Temps DP en mode greedy (abscisse) par rapport à Z3 Optimize sur MaxSAT (ordonnée), instance par instance. La diagonale $y = x$ sépare les instances où le DP est plus rapide (au-dessus) de celles où Z3 l'emporte (en-dessous). Trois instances se trouvent au-dessus de la diagonale, toutes de très petite taille ($n \leq 30$, $m \leq 60$).	32
5.2	ps-width obtenue par les modes greedy (vert) et linear (rouge) sur les 25 instances où l'écart est le plus marqué, parmi les 40 instances où les deux modes complètent. Les instances sont classées par ratio $\text{psw}_{\text{linear}}/\text{psw}_{\text{greedy}}$ (valeur indiquée à droite de chaque ligne).	33
5.3	Conflits Z3 par rapport à la ps-width effective (mode greedy), instance par instance. Chaque famille est représentée par une couleur distincte.	34
5.4	Temps de la requête de consistance (CO) sur DAG compilé (bleu) face à une décision SAT de Z3 (rouge), sur une instance représentative par famille. Le facteur d'accélération est indiqué à droite. Échelle logarithmique en abscisse.	35
5.5	À gauche : coût total du pipeline DP+requêtes en fonction du nombre N de requêtes (trait plein), comparé à N appels Z3 froids (pointillés), pour une instance représentative par famille. À droite : distribution cumulée du seuil N^* sur les instances où il est fini, médiane à 120.	36

1

Introduction

1.1 Contexte et motivation

La satisfaisabilité propositionnelle (SAT) occupe une place centrale en informatique théorique et appliquée depuis Cook [1], qui en a établi la NP-complétude, et Karp [2], qui a tiré de cette propriété une vaste famille de réductions polynomiales et fait de SAT le problème de référence de la théorie de la complexité. Malgré cette intractabilité théorique dans le cas général, les solveurs modernes parviennent à traiter des instances industrielles comportant des millions de variables et de clauses, grâce notamment à l'algorithme d'apprentissage de clauses guidé par les conflits (CDCL) [6, 5], à la propagation unitaire (BCP) et à des heuristiques de branchement sophistiquées. Cette efficacité pratique a conduit à une adoption massive de SAT dans des domaines aussi variés que la vérification formelle de circuits électroniques, la planification automatique, la synthèse de programmes et la cryptanalyse.

Ce succès a naturellement suscité un intérêt croissant pour des variantes plus expressives : le *comptage de modèles* (#SAT) et la *maximisation de clauses satisfaites* (MaxSAT). Le problème #SAT consiste à déterminer le nombre exact d'affectations satisfaisant une formule donnée en forme normale conjonctive (CNF). Il généralise la décision de satisfaisabilité en quantifiant la cardinalité de l'ensemble des modèles, et intervient naturellement dans le raisonnement probabiliste, l'inférence dans les réseaux bayésiens et la vérification probabiliste [10]. Le problème MaxSAT, quant à lui, cherche une affectation qui satisfait le plus grand nombre possible de clauses, même lorsque la formule est globalement insatisfaisable. Il modélise des problèmes d'optimisation combinatoire à contraintes souples (ordonnancement, configuration, bioinformatique) et constitue un cadre unifiant pour de nombreux problèmes NP-difficiles.

Ces deux variantes sont strictement plus difficiles que SAT : #SAT est #P-complet [3] et MaxSAT est NP-difficile. Cette complexité accrue justifie le développement d'approches algorithmiques spécialisées.

Une stratégie complémentaire à la résolution directe consiste à transformer hors-ligne la formule en une représentation structurée, typiquement un graphe orienté acyclique de type d-DNNF, sur laquelle un large éventail de requêtes (comptage, satisfaisabilité, équivalence) deviennent polynomiales. Cette *compilation de connaissances* [10] amortit le coût hors-ligne sur de multiples interrogations ultérieures, et est devenue le standard pour l'inférence probabiliste exacte et la vérification quantitative. Elle constitue le second pilier de ce mémoire, aux côtés de la résolution paramétrée par la structure de la formule.

1.2 Approche structurelle et décomposition arborescente

Plusieurs familles d’algorithmes ont émergé pour résoudre #SAT et MaxSAT : solveurs généraux fondés sur DPLL (Davis-Putnam-Logemann-Loveland) et CDCL, outils SMT (*satisfiability modulo theories*) comme Z3 [9], solveurs de comptage avec cache de composantes comme sharpSAT [7], ou encore la compilation de connaissances déjà évoquée. Ce travail combine deux perspectives complémentaires : la *complexité paramétrée structurelle*, qui exploite la structure du graphe d’incidence (graphe biparti reliant chaque variable aux clauses où elle apparaît) pour résoudre #SAT et MaxSAT en temps polynomial dès que le paramètre est borné ; et la *compilation de connaissances*, qui transforme cette même structure en un objet polynomialement interrogeable.

L’idée fondatrice est que la difficulté d’une instance ne dépend pas uniquement de sa taille $n + m$ (variables et clauses), mais de paramètres structurels liés à la façon dont variables et clauses interagissent. Si ces paramètres sont petits, des algorithmes par *programmation dynamique sur décomposition arborescente* résolvent #SAT et MaxSAT en temps polynomial en $n + m$, l’explosion exponentielle étant confinée à ce seul paramètre.

Parmi ces paramètres, la *treewidth* du graphe d’incidence de la formule est le plus étudié [13, 14]. Des résultats plus récents, fondés sur la *mim-width* [15] et la *ps-width* [18], étendent la tractabilité à des classes de formules strictement plus larges. L’algorithme de Saether, Telle et Vatshelle [18] résout simultanément #SAT et MaxSAT en temps $O(k^3 \cdot m \cdot (m + n))$, où k est la *ps-width* de la décomposition. Il constitue le premier pilier de ce mémoire de bachelor. Son extension par Bova, Capelli, Mengel et Slivovsky [17] en forme le second : ces auteurs l’instrumentent pour produire en parallèle un DAG d-DNNF, faisant ainsi le pont avec la compilation de connaissances.

1.3 Objectifs de ce travail

Ce mémoire de bachelor poursuit deux objectifs complémentaires.

Le premier est l’**implémentation complète** de l’algorithme de Saether, Telle et Vatshelle [18] en langage C. Cette implémentation comprend le parseur de formules DIMACS CNF, trois stratégies de construction de l’arbre de décomposition (dont l’heuristique GreedyOrder décrite dans le papier), les trois procédures de l’algorithme (calcul bottom-up des PS-sets, propagation top-down, programmation dynamique), ainsi que les structures de données sous-jacentes (bitsets 64-bit, trie binaire, tables DP).

Le second objectif est l’**extension à la compilation de connaissances** fondée sur le résultat de Bova, Capelli, Mengel et Slivovsky [17]. Leur algorithme s’obtient comme une instrumentation de la programmation dynamique précédente et produit, à coût marginal, un DAG d-DNNF de taille polynomiale en la *ps-width* et la taille de la formule. Sur le DAG produit, l’ensemble des requêtes que la carte de compilation de Darwiche et Marquis [10] rend polynomiales sur d-DNNF est implémenté.

Une **évaluation empirique** accompagne ces deux volets, sur des instances synthétiques conçues pour contrôler la *ps-width*, des formules aléatoires k -SAT et des circuits de multiplication encodés par la transformation de Tseytin. Ces tests mesurent l’impact de la *ps-width* sur les performances et délimitent les régimes dans lesquels la décomposition structurelle apporte un avantage décisif.

2

État de l'art

Ce chapitre positionne notre travail dans la littérature. Nous présentons successivement les problèmes SAT, #SAT et MaxSAT, les principales familles d'algorithmes de résolution, les paramètres de décomposition structurelle qui gouvernent leur complexité, l'algorithme de Saether, Telle et Vatschelle sur lequel repose notre implémentation, et la compilation de connaissances CNF $\rightarrow$ d-DNNF par Bova, Capelli, Mengel et Slivovsky qui en constitue l'extension naturelle.

2.1 SAT, #SAT et MaxSAT

2.1.1 Le problème SAT

Le problème de satisfaisabilité propositionnelle (SAT) demande, étant donnée une formule F en forme normale conjonctive (CNF), s'il existe une affectation des variables qui la satisfait. Rappelons qu'une formule CNF est une conjonction de clauses, chaque clause étant une disjonction de littéraux (variable ou sa négation). Une affectation satisfaisante est appelée *modèle* de F .

SAT fut le premier problème formellement démontré NP-complet par Cook [1], puis Karp en étendit l'applicabilité en montrant que de nombreux problèmes combinatoires classiques s'y réduisent en temps polynomial [2]. Malgré cette intractabilité théorique, les solveurs industriels modernes traitent couramment des instances de plusieurs millions de variables et clauses, grâce à l'algorithme CDCL [4, 5] décrit à la section 2.2.1.

2.1.2 #SAT : comptage de modèles

Le problème #SAT (*model counting*) demande non plus si un modèle existe, mais *combien* il en existe. Ce problème est #P-complet [3], classe de complexité qui capture le comptage associé aux décisions NP. Le comptage reste #P-complet même sur des fragments syntaxiques pour lesquels la décision est facile (2-CNF, Horn).

#SAT intervient dans l'inférence probabiliste sur des réseaux bayésiens [10], le model checking quantitatif et l'analyse de fiabilité de systèmes. Sa résolution exacte demande d'explorer l'intégralité de l'espace des affectations.

2.1.3 MaxSAT

Le problème MaxSAT cherche une affectation qui maximise le nombre de clauses satisfaites, même si la formule est globalement insatisfaisable. Il est NP-difficile et modélise des problèmes

d'optimisation à contraintes souples en ordonnancement, configuration ou bioinformatique. Sa variante *Weighted MaxSAT*, où chaque clause porte un poids et l'on maximise la somme des poids satisfaits, est celle que résout l'algorithme étudié dans ce mémoire.

2.2 Algorithmes de résolution

2.2.1 DPLL et CDCL

L'architecture CDCL (*Conflict-Driven Clause Learning*) étend l'algorithme historique DPLL (Davis-Putnam-Logemann-Loveland) par l'*apprentissage de clauses* : à chaque conflit, le solveur extrait une nouvelle clause qui résume la cause de l'échec et qu'il ajoute à la formule. Popularisée par zChaff [4] puis MiniSAT [5], CDCL est aujourd'hui le standard industriel et traite couramment des instances de plusieurs millions de variables.

2.2.2 Extensions pour #SAT et MaxSAT

Pour le comptage de modèles, les solveurs étendent DPLL par le *cache de composantes* : lorsque l'affectation partielle déconnecte le graphe d'incidence, les comptes des composantes disjointes sont calculés indépendamment puis multipliés. Ce mécanisme, popularisé par *sharpSAT* [7], est repris par D4 [8] qui compile en plus la formule vers d-DNNF. Pour la maximisation, les solveurs MaxSAT modernes reposent sur des approches *core-guided*, qui résolvent itérativement des noyaux insatisfaisables.

2.2.3 Compilation de connaissances

Une approche orthogonale à la résolution en ligne, la *compilation de connaissances*, transforme hors-ligne la formule en une représentation polynomialement interrogeable et fait l'objet de la section 2.5.

Toutes ces approches ont en commun de ne pas exploiter directement la structure topologique de la formule. La section suivante présente une famille d'algorithmes qui en fait le paramètre central.

2.3 Décompositions arborescentes et paramètres de largeur

Pour les formules issues du monde réel, la difficulté pratique dépend moins de la taille brute que de la structure de connectivité du *graphe d'incidence* de la formule : il s'agit du graphe biparti $I(F)$ dont les sommets sont d'un côté les variables $\text{var}(F)$, de l'autre les clauses $\text{cla}(F)$, avec une arête entre une variable x et une clause C chaque fois que x apparaît dans C . Plusieurs paramètres ont été proposés pour quantifier cette structure et fournir des algorithmes de programmation dynamique tractables quand le paramètre est borné.

Le plus étudié est la *treewidth*, introduite par Robertson et Seymour dans leur théorie des mineurs de graphes : on représente le graphe par une famille de sous-ensembles de sommets (*sacs*) connectés en arbre, et la largeur de la décomposition est la taille du plus grand sac diminuée de un. Samer et Szeider [13] puis Slivovsky et Szeider [14] ont obtenu des algorithmes de DP pour #SAT paramétrés par la *treewidth* du graphe d'incidence. Toutefois, la *treewidth* n'est bornée que sur des graphes globalement clairsemés, ce qui en limite la portée sur les classes denses.

Pour dépasser cette limitation, des paramètres plus généraux ont été introduits, dont la *mim-width* (*maximum induced matching width*) de Vatschelle [15]. Elle se définit via une décomposition arborescente du graphe : à chaque coupe, on mesure la taille du plus grand *couplage induit* traversant cette coupe (un couplage induit étant un ensemble d'arêtes deux à deux non adjacentes,

sans autre arête du graphe entre leurs extrémités) ; la *mim-width* est le minimum sur toutes les décompositions du maximum de cette quantité. Elle reste bornée sur des classes à *treewidth* non bornée (graphes d'intervalles ou de permutation, par exemple). En contrepartie, calculer la *mim-width* est intractable : Bergougnoux, Bonnet et Duron [16] ont montré que le problème est paraNP-complet, c'est-à-dire NP-difficile même pour des graphes de *mim-width* bornée par une constante. Cela impose le recours à des heuristiques dans toute mise en oeuvre pratique.

2.4 ps-width et l'algorithme de Saether-Telle-Vatshelle

Saether, Telle et Vatshelle [18] introduisent la *ps-width* (*projection-satisfiability width*), un paramètre défini directement sur les formules CNF à partir d'une *branch decomposition* de la formule. Il domine la *mim-width* du graphe d'incidence : pour des clauses de taille bornée, une *mim-width* bornée entraîne une *ps-width* polynomiale en le nombre de clauses, ce qui étend strictement les classes tractables au-delà de la *treewidth*. Les auteurs établissent que #SAT et MaxSAT se résolvent par programmation dynamique sur la décomposition en temps $O(k^3 \cdot m \cdot (m + n))$, ramené à $O(k^2 \cdot m \cdot (m + n))$ dans le cas linéaire.

Comme minimiser la *ps-width* est intractable, ils proposent l'heuristique gloutonne *GreedyOrder* et valident l'algorithme sur trois familles synthétiques d'instances à *ps-width* contrôlée : **sharpSAT** et les solveurs MaxSAT modernes restent plus rapides sur la plupart des entrées, mais sur les familles construites pour favoriser le DP leur prototype les bat nettement [18]. La définition formelle du paramètre, la description des trois procédures de l'algorithme et le fonctionnement de l'heuristique font l'objet du chapitre 3 ; les familles d'instances servent de banc de test au chapitre 5.

2.5 Compilation de connaissances CNF $\rightarrow$ d-DNNF

La *compilation de connaissances* (*knowledge compilation*, KC) transforme une formule CNF hors-ligne en une représentation structurée, typiquement un graphe orienté acyclique (DAG), sur laquelle certaines requêtes (comptage, conditionnement, projection partielle) deviennent polynomiales. Le coût de compilation est amorti dès qu'une même base est interrogée plusieurs fois ; la compilation est aujourd'hui un outil standard de l'inférence probabiliste exacte. Cette section présente la hiérarchie des langages cibles, les requêtes tractables sur d-DNNF, puis le résultat de Bova, Capelli, Mengel et Slivovsky (BCMS 2015) sur lequel repose l'extension KC de notre implémentation.

2.5.1 Langages cibles

Darwiche et Marquis [10] ont dressé la *carte de compilation* qui classe les représentations selon deux axes : la *succinctitude* (taille relative) et la *tractabilité* des requêtes supportées en temps polynomial. La hiérarchie pertinente pour notre travail va de la NNF (*Negation Normal Form* : DAG dont les feuilles sont des littéraux et les noeuds internes des ET et OU) à la DNNF (*Decomposable NNF* : enfants de tout ET à portées variables disjointes), puis au **d-DNNF** (*Deterministic DNNF* : enfants de tout OU à ensembles de modèles disjointes). Cette dernière propriété rend le comptage #SAT polynomial par un parcours qui remplace OU par somme et ET par produit, et fait du d-DNNF le langage cible de notre implémentation. Le SDD (*Sentential Decision Diagram* [11]) est un d-DNNF dont la structure est dictée par un *vtree*, un arbre binaire qui fixe l'ordre des variables aux feuilles. L'OBDD (*Ordered Binary Decision Diagram* [12]) est le précurseur historique. Ces deux variantes sont plus restrictives que d-DNNF et ne sont pas utilisées dans ce travail.

Au sein de la classe d-DNNF, deux sous-langages distincts se sont développés en parallèle. Les compilateurs DPLL-based, dont **c2d** puis **D4** [8], produisent des *decision-DNNF* via une recherche CDCL avec cache de composantes : les noeuds OU correspondent à des branchements sur une variable et la taille du DAG résultant n'est pas bornée a priori en termes de paramètres structurels de la formule. Les *d-DNNF structurés*, à l'inverse, imposent un schéma global (typiquement une branch decomposition fixée en amont) qui structure les noeuds AND et permet d'établir des bornes théoriques sur la taille de la représentation. La construction de Bova, Capelli, Mengel et Slivovsky (section 2.5.3) appartient à cette seconde famille et fournit une borne théorique sur la taille du DAG paramétrée par la ps-width.

2.5.2 Requêtes tractables sur d-DNNF

Darwiche et Marquis [10] établissent que la classe d-DNNF rend polynomiales six requêtes standard : **CO** (*consistency*), **VA** (*validity*), **CE** (*clausal entailment*), **IM** (*implicant check*), **CT** (*model counting*, le #SAT) et **ME** (*model enumeration*, énumération avec délai polynomial entre modèles consécutifs). Le DAG produit par la construction de la section 2.5.3 satisfait les hypothèses requises et ces six requêtes y sont donc directement disponibles. Leur algorithmique précise est décrite au chapitre 3.

2.5.3 Compilation explicite pilotée par la ps-width : BCMS 2015

Bova, Capelli, Mengel et Slivovsky [17] prolongent l'algorithme de Saether, Telle et Vatshelle en montrant que toute formule CNF F dotée d'une branch decomposition de ps-width k peut être compilée en un d-DNNF structuré de taille

$$|D| \leq 7 \cdot k^3 \cdot (n + m),$$

en temps polynomial en $n + m$. À la différence des compilateurs DPLL-based (**c2d**, **D4**), la construction n'effectue aucune recherche : la décomposition fournit le squelette du DAG, le DP fournit son contenu, et la borne $7 \cdot k^3 \cdot (n + m)$ donne une garantie de taille théorique paramétrée par la ps-width, absente des compilateurs top-down. La construction détaillée du DAG est décrite au chapitre 3.

3

Cadre théorique

Les sections 3.1 et 3.2 fixent les notations et définissent la branch decomposition puis la ps-width. La section 3.3 expose la programmation dynamique de Saether, Telle et Vatshelle [18] qui résout #SAT et MaxSAT en temps polynomial pour ps-width bornée, et la section 3.4 l’heuristique gloutonne GreedyOrder qui lui fournit une décomposition. Bova, Capelli, Mengel et Slivovsky [17] (section 3.5) instrumentent ce DP pour produire un d-DNNF structuré, sur lequel la section 3.6 décrit les six requêtes polytimes.

3.1 Notations et préliminaires

On considère une formule F en CNF, vue comme un multi-ensemble de clauses, chaque clause étant un ensemble de littéraux (une variable ou sa négation). On note $\text{var}(F)$ l’ensemble des variables apparaissant dans F et $\text{cla}(F)$ son ensemble de clauses. Une affectation d’un ensemble de variables X est une fonction $\tau : X \rightarrow \{0, 1\}$, étendue aux littéraux par $\tau(\neg x) = 1 - \tau(x)$. Elle satisfait une clause C s’il existe un littéral $\ell \in C$ tel que $\tau(\ell) = 1$, et satisfait la formule F si elle satisfait toutes les clauses de $\text{cla}(F)$.

Pour $\mathcal{C} \subseteq \text{cla}(F)$ et $X \subseteq \text{var}(F)$, la *sous-formule induite* $F_{\mathcal{C}, X}$ s’obtient en retirant de F toute clause hors de $\mathcal{C}$, puis tout littéral dont la variable n’appartient pas à X . Cette construction décrira les sous-formules de part et d’autre d’une coupe de la décomposition (section 3.2).

Le graphe d’incidence $I(F)$ a été défini au chapitre 2, section 2.3. Ses sommets sont les variables et clauses de F , avec une arête entre x et C si et seulement si x apparaît dans C . On suit l’hypothèse de Saether, Telle et Vatshelle [18] en supposant $I(F)$ connexe : sinon le problème se résout indépendamment sur chaque composante connexe.

3.2 Branch decomposition et ps-width

3.2.1 Branch decomposition d’une formule

Une *branch decomposition* d’une formule F est un couple (T, δ) où T est un arbre binaire enraciné et δ une bijection entre les feuilles de T et l’ensemble $\text{var}(F) \cup \text{cla}(F)$ [18]. Chaque variable et chaque clause de F apparaît ainsi exactement une fois comme étiquette d’une feuille. Pour un noeud non-feuille v de T , on note $\delta(v)$ l’ensemble des étiquettes des feuilles du sous-arbre enraciné en v :

$$\delta(v) = \{ \delta(l) : l \text{ feuille du sous-arbre enraciné en } v \}.$$

Le noeud v partage $\text{var}(F) \cup \text{cla}(F)$ en deux, $\delta(v)$ et son complément. Cette partition est appelée la *coupe* associée à v .

À chaque noeud v sont associées deux sous-formules. En écrivant $\delta(v) = \mathcal{C} \cup X$ avec $\mathcal{C} \subseteq \text{cla}(F)$ et $X \subseteq \text{var}(F)$, on pose

$$F_v = F_{\bar{\mathcal{C}}, X} \quad \text{et} \quad F_{\bar{v}} = F_{\mathcal{C}, \bar{X}},$$

où $\bar{\mathcal{C}} = \text{cla}(F) \setminus \mathcal{C}$ et $\bar{X} = \text{var}(F) \setminus X$, et où la notation $F_{\mathcal{C}, X}$ a été fixée en section 3.1. L'asymétrie est volontaire : la sous-formule F_v rassemble les clauses extérieures à la coupe restreintes aux variables locales, et décrit précisément ce qu'une affectation des seules variables de X peut satisfaire parmi les clauses du complément.

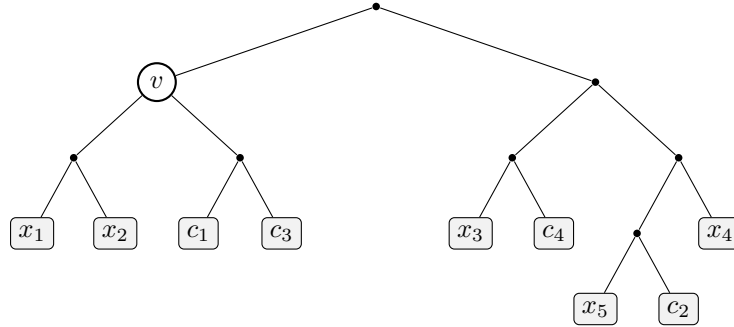


FIGURE 3.1 – Une branch decomposition (T, δ) d'une formule F sur cinq variables et quatre clauses, reprise de la Figure 2 de [18]. La formule a pour variables $\text{var}(F) = \{x_1, x_2, x_3, x_4, x_5\}$ et pour clauses $c_1 = \{x_1, x_2\}$, $c_2 = \{x_1, \neg x_2, x_3\}$, $c_3 = \{x_2, \neg x_4, x_5\}$, $c_4 = \{x_2, x_3, x_5\}$. Le noeud v marqué définit la coupe $\delta(v) = \{x_1, x_2, c_1, c_3\}$, soit $\mathcal{C} = \{c_1, c_3\}$ et $X = \{x_1, x_2\}$. Les deux sous-formules croisées sont $F_v = \{\{x_1, \neg x_2\}, \{x_2\}\}$ (versions de c_2 et c_4 restreintes aux variables de X) et $F_{\bar{v}} = \{\emptyset, \{\neg x_4, x_5\}\}$ (versions de c_1 et c_3 restreintes aux variables de $\bar{X}$).

Quand tous les noeuds internes de T se trouvent sur un même chemin (autrement dit, chaque noeud interne a au moins une feuille pour enfant), on parle de *décomposition linéaire*. Ce cas particulier admet une analyse de complexité plus fine, traitée en section 3.3.5.

3.2.2 Coupes et PS-sets

Soient $\mathcal{C} \subseteq \text{cla}(F)$ et $X \subseteq \text{var}(F)$. L'ensemble $S = \mathcal{C} \cup X$ est une *coupe* de F . Chaque noeud non-feuille v d'une branch decomposition en définit une via $\delta(v)$. Une coupe sépare F en quatre sous-formules induites $F_{\mathcal{C}, X}$, $F_{\bar{\mathcal{C}}, X}$, $F_{\mathcal{C}, \bar{X}}$ et $F_{\bar{\mathcal{C}}, \bar{X}}$, chacune obtenue en restreignant clauses et variables aux sous-ensembles qui composent S ou son complément.

Pour une affectation τ d'un sous-ensemble de variables, on note $\text{sat}(F, \tau)$ l'ensemble inclusion-maximal des clauses de F satisfaites par τ . Un sous-ensemble $\mathcal{C} \subseteq \text{cla}(F)$ est dit *projection-satisfiable* dans F s'il existe une affectation τ portant sur l'ensemble complet $\text{var}(F)$ telle que $\text{sat}(F, \tau) = \mathcal{C}$. On dit alors que $\mathcal{C}$ est une *projection* de F par τ .

La famille des sous-ensembles projection-satisfiables de F se note

$$\text{PS}(F) = \{ \text{sat}(F, \tau) : \tau \text{ est une affectation de } \text{var}(F) \}.$$

Sa cardinalité $|\text{PS}(F)|$ est la *ps-value* de F . C'est à partir de cette quantité, calculée sur les sous-formules F_v et $F_{\bar{v}}$ associées aux noeuds d'une branch decomposition, que la section 3.2.3 définira la ps-width.

3.2.3 ps-value et ps-width

Pour un noeud v d'une branch decomposition (T, δ) , la *ps-value* de la coupe associée se définit par

$$\text{ps}(\delta(v)) = \max(|\text{PS}(F_v)|, |\text{PS}(F_{\bar{v}})|).$$

La *ps-width* de la décomposition est la plus grande de ces ps-values sur tous les noeuds de T :

$$\text{psw}(T, \delta) = \max \{ \text{ps}(\delta(v)) : v \in T \}.$$

La *ps-width* de la formule F est le minimum atteint sur toutes ses branch decompositions :

$$\text{psw}(F) = \min \{ \text{psw}(T, \delta) : (T, \delta) \text{ branch decomposition de } F \}.$$

Pour une clause c et un ensemble de variables Y , on note $c|_Y$ la clause obtenue en ne gardant que les littéraux dont la variable est dans Y : $c|_Y = \{ \ell \in c : \text{var}(\ell) \in Y \}$. Détaillons le calcul de $|\text{PS}(F_v)|$ sur l'exemple de la Figure 3.1. La sous-formule F_v contient les versions restreintes de c_2 et c_4 aux variables de $X = \{x_1, x_2\}$:

$$D_1 = c_2|_X = \{x_1, \neg x_2\}, \quad D_2 = c_4|_X = \{x_2\}.$$

Les quatre affectations possibles sur $\text{var}(F_v) = \{x_1, x_2\}$ donnent :

x_1	x_2	D_1	D_2	$\text{sat}(F_v, \tau)$
0	0	satisfaite	non satisfaite	$\{D_1\}$
0	1	non satisfaite	satisfaite	$\{D_2\}$
1	0	satisfaite	non satisfaite	$\{D_1\}$
1	1	satisfaite	satisfaite	$\{D_1, D_2\}$

Les quatre affectations produisent trois ensembles satisfaits distincts, d'où $\text{PS}(F_v) = \{\{D_1\}, \{D_2\}, \{D_1, D_2\}\}$ et $|\text{PS}(F_v)| = 3$. Du côté de $F_{\bar{v}} = \{\emptyset, \{\neg x_4, x_5\}\}$, la clause vide n'est jamais satisfaite, et les affectations sur $\{x_4, x_5\}$ qui satisfont $\{\neg x_4, x_5\}$ donnent $\text{sat} = \{c_3|_{\bar{X}}\}$, les autres donnent $\text{sat} = \emptyset$. D'où $\text{PS}(F_{\bar{v}}) = \{\emptyset, \{c_3|_{\bar{X}}\}\}$ et $|\text{PS}(F_{\bar{v}})| = 2$. La ps-value de la coupe vaut donc $\text{ps}(\delta(v)) = \max(3, 2) = 3$.

La programmation dynamique de la section 3.3 manipule plus naturellement les clauses originales de F que leurs versions restreintes. Le papier introduit pour cela une variante de notation : $\text{PS}'(F_v)$ désigne la famille des sous-ensembles $\mathcal{D} \subseteq \text{cla}(F) \setminus \delta(v)$ dont la restriction aux variables de F_v appartient à $\text{PS}(F_v)$, et $\text{sat}'(F_v, \tau)$ est définie de manière analogue à partir de $\text{sat}(F_v, \tau)$. Sur notre exemple, $\text{PS}'(F_v) = \{\{c_2\}, \{c_4\}, \{c_2, c_4\}\}$: chaque élément de $\text{PS}(F_v)$ est remplacé par le sous-ensemble correspondant de clauses originales de F . La correspondance entre PS et PS' est bijective : $|\text{PS}'(F_v)| = |\text{PS}(F_v)|$ et la ps-value est inchangée.

Saether, Telle et Vatshelle établissent par ailleurs (Lemme 1 de [18]) que pour des clauses de taille bornée, $|\text{PS}(F)|$ reste polynomial en le nombre de clauses dès que la taille maximale d'un couplage induit du graphe d'incidence $I(F)$ est bornée : la ps-width est donc un paramètre tractable sur les classes de formules dont la mim-width (définie en section 2.3) est bornée.

3.3 Programmation dynamique pour #SAT et MaxSAT

3.3.1 Stratégie d'ensemble

Le calcul de #SAT et MaxSAT exploite la structure récursive de la branch decomposition (T, δ) . À chaque noeud v de T , la coupe $\delta(v)$ délimite les sous-formules F_v et $F_{\bar{v}}$, et toute

affectation τ de $\text{var}(F)$ se décompose en sa restriction $\tau|_v$ aux variables de F_v et sa restriction $\tau|_{\bar{v}}$ aux variables de $F_{\bar{v}}$. Comme une clause peut être satisfaite par des littéraux situés des deux côtés de la coupe, la programmation dynamique de Saether, Telle et Vatshelle [18] retient à chaque coupe l'ensemble des clauses déjà satisfaites par chaque moitié, ce qui permet de combiner les contributions des deux côtés sans énumérer toutes les affectations. Les PS-sets introduits en section 3.2.3 fournissent l'espace d'états adapté : la ps-width k borne leur cardinal à chaque coupe, et les boucles internes du DP restent polynomiales en k .

L'algorithme s'organise en trois procédures successives. La *Procédure 1* calcule, par parcours bottom-up de T , les ensembles $\text{PS}'(F_v)$. La *Procédure 2* calcule, par parcours top-down, les ensembles duaux $\text{PS}'(F_{\bar{v}})$. Ces deux passes sont des pré-traitements obligatoires : la *Procédure 3*, qui effectue le DP proprement dit en remontant l'arbre, a besoin en chaque noeud des PS-sets des deux côtés de la coupe pour indexer ses tables. La valeur cherchée se lit ensuite à la racine, où la table se réduit à une unique cellule. Les sections 3.3.2 à 3.3.5 en formalisent successivement chaque étape.

3.3.2 Procédure 1 : calcul bottom-up des $\text{PS}'(F_v)$

La Procédure 1 calcule $\text{PS}'(F_v)$ pour tous les noeuds de la branch decomposition par parcours bottom-up de T . À chaque noeud, le résultat dépend uniquement des PS-sets calculés aux deux enfants, et la récursion atteint les feuilles, où le calcul est explicite.

Aux feuilles, $\text{var}(F_\ell)$ contient au plus une variable et $\text{PS}'(F_\ell)$ se construit directement. Si ℓ est étiquetée par une variable x , les deux affectations $\tau(x) = 1$ et $\tau(x) = 0$ produisent

$$\text{sat}'(F_\ell, \tau=1) = \{c \in \text{cla}(F) : x \in c\}, \quad \text{sat}'(F_\ell, \tau=0) = \{c \in \text{cla}(F) : \neg x \in c\},$$

qui sont les deux éléments candidats de $\text{PS}'(F_\ell)$. Le cardinal de $\text{PS}'(F_\ell)$ vaut 1 ou 2 selon que ces deux ensembles coïncident ou non. Si ℓ est étiquetée par une clause c , alors $\text{var}(F_\ell) = \emptyset$, l'unique affectation ε (sur l'ensemble vide de variables) ne satisfait aucune clause restreinte à $\emptyset$, et $\text{PS}'(F_\ell) = \{\emptyset\}$.

Pour un noeud interne v d'enfants c_1 et c_2 , le PS-set se reconstruit récursivement par

$$\text{PS}'(F_v) = \{ (C_1 \cup C_2) \cap \text{cla}(F_v) : C_1 \in \text{PS}'(F_{c_1}), C_2 \in \text{PS}'(F_{c_2}) \}.$$

La formule reflète la décomposition $\text{var}(F_v) = \text{var}(F_{c_1}) \sqcup \text{var}(F_{c_2})$: une affectation τ de $\text{var}(F_v)$ se sépare en $\tau_1 = \tau|_{\text{var}(F_{c_1})}$ et $\tau_2 = \tau|_{\text{var}(F_{c_2})}$, et une clause $c \in \text{cla}(F_v)$ est satisfaite par τ si et seulement si τ_1 ou τ_2 y satisfait un littéral. L'intersection avec $\text{cla}(F_v) = \text{cla}(F) \setminus \delta(v)$ écarte les clauses étiquetant les feuilles du sous-arbre, qui sortent du domaine de F_v .

Procédure 1 Génération de $\text{PS}'(F_v)$

Require: $\text{PS}'(F_{c_1})$, $\text{PS}'(F_{c_2})$ pour les enfants c_1, c_2 de v

Ensure: $\text{PS}'(F_v)$

- 1: $L \leftarrow \emptyset$
 - 2: **for all** $(C_1, C_2) \in \text{PS}'(F_{c_1}) \times \text{PS}'(F_{c_2})$ **do**
 - 3: ajouter $(C_1 \cup C_2) \cap \text{cla}(F_v)$ à L
 - 4: **end for**
 - 5: **return** L
-

Théorème 3.1 (Théorème 1 de [18]). *Soit F une formule CNF avec $n = |\text{var}(F)|$ et $m = |\text{cla}(F)|$, et soit (T, δ) une branch decomposition de F de ps-width k . L'ensemble des $\text{PS}'(F_v)$ et des $\text{PS}'(F_{\bar{v}})$ pour tous les noeuds v de T se calcule en temps $O(k^2 m (m + n))$.*

3.3.3 Procédure 2 : calcul top-down des $\text{PS}'(F_{\bar{v}})$

La Procédure 2 calcule $\text{PS}'(F_{\bar{v}})$ pour tous les noeuds v de T par parcours top-down. Elle remplit le rôle symétrique de la Procédure 1 sur l'autre côté de chaque coupe : la récursion descend de la racine vers les feuilles, et chaque calcul à un noeud non-racine combine deux PS-sets déjà disponibles, celui du frère s produit par la Procédure 1 et celui du parent p obtenu plus haut dans la descente.

À la racine r , $\text{var}(F_{\bar{r}}) = \emptyset$ et l'unique affectation ε ne satisfait aucune clause restreinte à $\emptyset$. On a donc $\text{PS}(F_{\bar{r}}) = \{\emptyset\}$, d'où le cas de base

$$\text{PS}'(F_{\bar{r}}) = \{\emptyset\}.$$

Pour un noeud non-racine v de frère s et de parent p , le PS-set s'obtient par

$$\text{PS}'(F_{\bar{v}}) = \{ (C_s \cup C_p) \cap \text{cla}(F_{\bar{v}}) : C_s \in \text{PS}'(F_s), C_p \in \text{PS}'(F_{\bar{p}}) \}.$$

La formule reflète la décomposition $\text{var}(F_{\bar{v}}) = \text{var}(F_s) \sqcup \text{var}(F_{\bar{p}})$: les variables hors de $\delta(v)$ proviennent soit du sous-arbre du frère ($\text{var}(F_s) \subseteq \delta(s)$), soit de l'extérieur du sous-arbre du parent ($\text{var}(F_{\bar{p}}) \subseteq \bar{\delta}(p)$). Une affectation de $\text{var}(F_{\bar{v}})$ se sépare donc en deux moitiés indépendantes, et l'intersection avec $\text{cla}(F_{\bar{v}}) = \text{cla}(F) \cap \delta(v)$ écarte les clauses étiquetant des feuilles extérieures au sous-arbre de v , qui sortent du domaine de $F_{\bar{v}}$.

Procédure 2 Génération de $\text{PS}'(F_{\bar{v}})$

Require: $\text{PS}'(F_s)$, $\text{PS}'(F_{\bar{p}})$ pour le frère s et le parent p de v

Ensure: $\text{PS}'(F_{\bar{v}})$

- 1: $L \leftarrow \emptyset$
 - 2: **for all** $(C_s, C_p) \in \text{PS}'(F_s) \times \text{PS}'(F_{\bar{p}})$ **do**
 - 3: ajouter $(C_s \cup C_p) \cap \text{cla}(F_{\bar{v}})$ à L
 - 4: **end for**
 - 5: **return** L
-

La Procédure 2 a la même structure que la Procédure 1 et s'analyse de la même manière, d'où l'énoncé conjoint du Théorème 3.1.

3.3.4 Procédure 3 : table DP $\text{Tab}_v(C, C')$

La Procédure 3 calcule la table de programmation dynamique en chaque noeud par parcours bottom-up de T . C'est l'étape qui exploite les PS-sets produits par les Procédures 1 et 2 pour obtenir, à la racine, la solution de MaxSAT et de #SAT.

Soit v un noeud de T et $(C, C') \in \text{PS}'(F_v) \times \text{PS}'(F_{\bar{v}})$. Une affectation τ de $\text{var}(F)$ rencontre l'expectation (C, C') si elle vérifie simultanément

$$(A) \text{ sat}'(F_v, \tau|_{\text{var}(F_v)}) = C,$$

$$(B) \text{ sat}'(F_{\bar{v}}, \tau|_{\text{var}(F_{\bar{v}})}) = C'.$$

La condition (A) fixe la trace que les variables intérieures à $\delta(v)$ laissent sur les clauses extérieures, et la condition (B) fixe la trace symétrique des variables extérieures sur les clauses intérieures. La table de programmation dynamique attachée à v est l'application

$$\text{Tab}_v : \text{PS}'(F_v) \times \text{PS}'(F_{\bar{v}}) \rightarrow \mathbb{N},$$

où $\text{Tab}_v(C, C')$ est le nombre maximal de clauses de $\delta(v)$ satisfaites par une affectation rencontrant l'expectation (C, C') . Comme les variables hors de $\delta(v)$ y satisfont exactement les clauses de C' , cette définition équivaut à la Constraint (1) de [18] :

$$\text{Tab}_v(C, C') = \max_{\tau} |(\text{sat}'(F, \tau) \cap \delta(v)) \cup C'|, \quad (3.1)$$

où τ parcourt les affectations de $\text{var}(F) \cap \delta(v)$ telles que $\text{sat}'(F_v, \tau) = C$.

Pour un noeud interne v d'enfants c_1 et c_2 , la valeur de $\text{Tab}_v(C_v, C'_v)$ se reconstruit à partir des tables filles Tab_{c_1} et Tab_{c_2} . Chaque table fille étant indexée par un couple, une mise à jour fait intervenir six indices au total. Une astuce due à Bui-Xuan, Telle et Vatschelle réduit la boucle de six à trois indices : on itère sur les triplets $(C_{c_1}, C_{c_2}, C'_v) \in \text{PS}'(F_{c_1}) \times \text{PS}'(F_{c_2}) \times \text{PS}'(F_{\bar{v}})$, et l'on dérive canoniquement

$$C'_{c_1} = (C_{c_2} \cup C'_v) \cap \delta(c_1), \quad C'_{c_2} = (C_{c_1} \cup C'_v) \cap \delta(c_2), \quad C_v = (C_{c_1} \cup C_{c_2}) \setminus \delta(v).$$

Ces trois identités expriment les relations entre les couples d'expectation qu'une même affectation rencontre aux coupes de c_1 , c_2 et v . La cellule $\text{Tab}_v(C_v, C'_v)$ se met alors à jour par le maximum des sommes $\text{Tab}_{c_1}(C_{c_1}, C'_{c_1}) + \text{Tab}_{c_2}(C_{c_2}, C'_{c_2})$ ainsi obtenues.

Procédure 3 Calcul de Tab_v pour un noeud interne v d'enfants c_1, c_2 (MaxSAT)

Require: $\text{Tab}_{c_1}, \text{Tab}_{c_2}$

Ensure: Tab_v

```

1:  $\text{Tab}_v(C, C') \leftarrow -1$  pour tout  $(C, C') \in \text{PS}'(F_v) \times \text{PS}'(F_{\bar{v}})$ 
2: for all  $(C_{c_1}, C_{c_2}, C'_v) \in \text{PS}'(F_{c_1}) \times \text{PS}'(F_{c_2}) \times \text{PS}'(F_{\bar{v}})$  do
3:    $C'_{c_1} \leftarrow (C_{c_2} \cup C'_v) \cap \delta(c_1)$ 
4:    $C'_{c_2} \leftarrow (C_{c_1} \cup C'_v) \cap \delta(c_2)$ 
5:    $C_v \leftarrow (C_{c_1} \cup C_{c_2}) \setminus \delta(v)$ 
6:    $t \leftarrow \text{Tab}_{c_1}(C_{c_1}, C'_{c_1}) + \text{Tab}_{c_2}(C_{c_2}, C'_{c_2})$ 
7:   if  $\text{Tab}_v(C_v, C'_v) < t$  then
8:      $\text{Tab}_v(C_v, C'_v) \leftarrow t$ 
9:   end if
10: end for
11: return  $\text{Tab}_v$ 
```

Pour #SAT, la cellule $\text{Tab}_v(C, C')$ compte plutôt le nombre d'affectations τ de $\text{var}(F) \cap \delta(v)$ telles que $\text{sat}'(F_v, \tau) = C$ et que toute clause de $\delta(v)$ soit satisfaite par τ ou figure dans C' . La Procédure 3 s'adapte de la manière suivante : la table s'initialise à 0 partout, les feuilles reçoivent les valeurs 0, 1 ou 2 données par la définition, et les lignes 6 et 7 du pseudocode sont remplacées par

$$\text{Tab}_v(C_v, C'_v) \leftarrow \text{Tab}_v(C_v, C'_v) + \text{Tab}_{c_1}(C_{c_1}, C'_{c_1}) \cdot \text{Tab}_{c_2}(C_{c_2}, C'_{c_2}).$$

La somme cumulée remplace le maximum, et le produit des deux tables filles dénombre les combinaisons compatibles à chaque itération.

Lemme 3.2 (Lemme 2 de [18]). *Soit F une formule CNF de m clauses et v un noeud interne d'une branch decomposition (T, δ) de ps-width k . La Procédure 3 calcule Tab_v vérifiant l'équation (3.1) en temps $O(k^3 m)$.*

3.3.5 Théorèmes 2 et 3

Une fois les trois procédures exécutées sur T , le résultat du DP se lit en une seule cellule à la racine. Comme noté en section 3.3.1, à la racine r on a $\text{PS}'(F_r) = \text{PS}'(F_{\bar{r}}) = \{\emptyset\}$, et la table Tab_r se réduit à l'unique cellule $\text{Tab}_r(\emptyset, \emptyset)$.

Théorème 3.3 (Théorème 2 de [18]). *Soit F une formule CNF de n variables et m clauses, et (T, δ) une branch decomposition de F de ps-width k . Le DP calcule MaxSAT et $\#SAT$ en temps $O(k^3 m(m+n))$, la valeur cherchée se lisant à la racine en $\text{Tab}_r(\emptyset, \emptyset)$.*

La validité vient de l'équation (3.1) : à la racine, $\text{cla}(F) \cap \delta(r) = \text{cla}(F)$, donc $\text{Tab}_r(\emptyset, \emptyset)$ vaut le maximum du nombre de clauses satisfiables par une affectation de $\text{var}(F)$ (MaxSAT). La variante additive de la Procédure 3 produit, au même endroit, le nombre d'affectations qui satisfont toutes les clauses ($\#SAT$). La complexité s'obtient en sommant le coût du pré-traitement (Théorème 3.1, $O(k^2 m(m+n))$) et celui de la Procédure 3 sur les $O(m+n)$ noeuds internes (Lemme 3.2, $O(k^3 m)$ par noeud).

Le facteur k^3 provient du triplet sur lequel boucle la Procédure 3 à chaque noeud interne. Sur une décomposition *linéaire* (chaque noeud interne a au moins un enfant feuille), le PS-set de cet enfant est de cardinal au plus 2, soit constant. Le triplet se réduit donc à $O(k^2)$ termes, et la borne globale tombe d'un facteur k .

Théorème 3.4 (Théorème 3 de [18]). *Sous les hypothèses du Théorème 3.3, si (T, δ) est une décomposition linéaire, alors MaxSAT et $\#SAT$ se résolvent en temps $O(k^2 m(m+n))$.*

Cette amélioration ne demande aucune modification de l'algorithme : la boucle de la Procédure 3 itère sur les tailles effectives des PS-sets des enfants, et l'un d'eux étant constant sur une décomposition linéaire, le compte d'itérations par noeud passe à $O(k^2)$. Les décompositions produites par l'heuristique GreedyOrder, présentée en section 3.4, sont précisément de ce type.

3.4 Heuristique GreedyOrder

L'algorithme du DP exposé en section 3.3 prend en entrée une branch decomposition (T, δ) . Pour minimiser le facteur k dans la complexité, il faudrait disposer de la ps-width minimale parmi toutes les branch decompositions de F . Calculer cette quantité est probablement intractable [18], mais aucune démonstration formelle n'en a été donnée. En pratique, le DP s'exécute donc sur une décomposition produite par une heuristique, sans garantie d'optimalité.

GreedyOrder est l'heuristique élémentaire proposée par le papier pour cet usage. Elle prend en entrée le graphe d'incidence $I(F)$ et produit un ordre linéaire σ sur ses sommets (variables et clauses confondues). À chaque étape, elle choisit comme prochain sommet celui ayant le plus grand nombre de voisins *déjà choisis*, en départageant les ex aequo par le plus petit degré total.

Procédure 4 GreedyOrder

Require: Graphe biparti $G = (V, E)$

Ensure: Ordre linéaire σ sur V

```

1:  $L \leftarrow \emptyset, R \leftarrow V, i \leftarrow 1$ 
2: for all  $v \in V$  do
3:    $Ldegree(v) \leftarrow 0$ 
4: end for
5: while  $R \neq \emptyset$  do
6:   choisir  $v$  : parmi les sommets de  $R$  de  $Ldegree$  maximal, prendre un de plus petit degré total
7:    $\sigma(i) \leftarrow v, i \leftarrow i + 1$ , transférer  $v$  de  $R$  vers  $L$ 
8:   for all  $w \in R$  tel que  $vw \in E$  do
9:      $Ldegree(w) \leftarrow Ldegree(w) + 1$ 
10:  end for
11: end while
12: return  $\sigma$ 
```

L'ordre σ se traduit en une branch decomposition linéaire (T, δ) : T est un arbre binaire dont les noeuds internes forment un chemin enraciné à une extrémité, et δ envoie la i -ème feuille rencontrée par un parcours en largeur partant de la racine sur le sommet $\sigma(i)$. La structure résultante est un *caterpillar* : chaque noeud interne a une feuille pour enfant, à l'exception du noeud le plus profond qui en a deux. C'est précisément la forme d'arbre où la borne $O(k^2 m(m+n))$ du Théorème 3.4 s'applique.

GreedyOrder n'offre aucune garantie théorique sur la ps-width de la décomposition produite : les auteurs la qualifient eux-mêmes de « very simple heuristic » et la présentent comme une simple preuve de concept. Son intérêt réside dans la simplicité de mise en oeuvre et dans les performances empiriques observées au chapitre 5.

3.5 Compilation de F en d-DNNF structuré (BCMS)

Bova, Capelli, Mengel et Slivovsky [17] (BCMS dans la suite) montrent qu'à partir de la programmation dynamique de Saether, on peut construire en parallèle un DAG d-DNNF qui calcule la formule F . Leur papier introduit pour cela un objet appelé *shape*, noté $\mathbf{S} = (S, S')$. Comme cette notion coïncide avec les paires (C, C') déjà utilisées en section 3.3 (mêmes ensembles sous-jacents, et même sémantique que la variante #SAT du DP de Saether, section 3.3.4), nous conservons la notation (C, C') dans tout ce qui suit pour préserver l'unité de lecture du chapitre. Les écarts ponctuels avec la nomenclature BCMS seront signalés explicitement.

3.5.1 Construction du DAG par récurrence

BCMS associe à chaque noeud v et chaque paire $(C, C') \in \mathbf{PS}'(F_v) \times \mathbf{PS}'(F_{\bar{v}})$ un d-DNNF $\varphi_v(C, C')$ construit récursivement à partir de ses enfants. La récurrence repose sur les mêmes dérivations à trois indices que la Procédure 3 (section 3.3.4), vues comme une relation de *génération* entre une paire-parent et un couple de paires-enfants : (C_1, C'_1) et (C_2, C'_2) *gènèrent* (C, C') si

$$C = (C_1 \cup C_2) \setminus \delta(v), \quad C'_1 = (C' \cup C_2) \cap \delta(c_1), \quad C'_2 = (C' \cup C_1) \cap \delta(c_2).$$

On note $G(C, C')$ l'ensemble des couples qui génèrent (C, C') en v , et $N_v(C, C')$ l'ensemble des affectations $\tau : \text{var}(F_v) \rightarrow \{0, 1\}$ satisfaisant les conditions de la variante #SAT du DP (section 3.3.4).

Aux feuilles de T , les ensembles $N_v(C, C')$ sont explicites et les fragments φ_v se réduisent à des littéraux ou des constantes. Pour une **feuille variable** $\delta(v) = \{x\}$, soient $\tau_x, \tau_{\neg x}$ les deux affectations de $\{x\}$ et $C_\ell = \text{sat}'(F_v, \tau_\ell)$ pour $\ell \in \{x, \neg x\}$. Les paires admises sont $(C_x, \emptyset)$ et $(C_{\neg x}, \emptyset)$, et l'on pose

$$\varphi_v(C_x, \emptyset) \equiv x, \quad \varphi_v(C_{\neg x}, \emptyset) \equiv \neg x.$$

Pour une **feuille clause** $\delta(v) = \{c\}$, l'unique affectation ε sur l'ensemble vide de variables ne satisfait aucune clause. Les paires admises sont $(\emptyset, \emptyset)$ et $(\emptyset, \{c\})$. Comme $\varepsilon \in N_v(\emptyset, \{c\})$ mais $\varepsilon \notin N_v(\emptyset, \emptyset)$, on pose

$$\varphi_v(\emptyset, \emptyset) \equiv \perp, \quad \varphi_v(\emptyset, \{c\}) \equiv \top.$$

Pour un noeud interne v d'enfants c_1 et c_2 , et une paire (C, C') en v , on combine les fragments des enfants via

$$\varphi_v(C, C') \equiv \bigvee_{((C_1, C'_1), (C_2, C'_2)) \in G(C, C')} \varphi_{c_1}(C_1, C'_1) \wedge \varphi_{c_2}(C_2, C'_2). \quad (3.2)$$

Un noeud AND conjugue les fragments des deux enfants pour chaque couple générateur, et tous ces noeuds AND alimentent un même noeud OR qui forme la racine de $\varphi_v(C, C')$. Les constantes $\perp$ et $\top$ sont propagées en cours de construction et les sous-termes triviaux supprimés.

Le d-DNNF $\varphi_v(C, C')$ ainsi obtenu a deux propriétés : ses variables sont contenues dans $\text{var}(F_v)$, et ses modèles sont exactement les affectations de $N_v(C, C')$. C'est le contenu du Lemme 4 de BCMS, énoncé en section 3.5.2.

À la racine r de T , l'unique paire est $(\emptyset, \emptyset)$, et la compilation de F s'obtient comme $\varphi_r(\emptyset, \emptyset)$ par évaluation de la récurrence (3.2) en remontant l'arbre. La correction de cette compilation est formalisée en section 3.5.2.

3.5.2 Propriétés du DAG produit

La construction décrite à la section 3.5.1 est correcte comme compilateur de F en d-DNNF structuré. Sa justification s'organise autour de trois lemmes du papier BCMS, que l'on énonce sans démonstration.

Lemme 3.5 (Lemme 4 de [17]). *Pour tout noeud v de T et toute paire (C, C') admise en v , le DAG $\varphi_v(C, C')$ est un d-DNNF dont les variables sont incluses dans $\text{var}(F_v)$. Une affectation $\tau : \text{var}(F_v) \rightarrow \{0, 1\}$ satisfait $\varphi_v(C, C')$ si et seulement si $\tau \in N_v(C, C')$.*

Pour énoncer la structuration, on associe à chaque noeud v un *vtree* de $\text{var}(F_v)$, noté $\text{vtree}(T, v)$. C'est le sous-arbre T_v dans lequel on supprime les feuilles étiquetées par une clause, puis dans lequel on contracte les arêtes pendantes pour restaurer la binarité (Figure 1 de [17]). Le résultat est une branch decomposition de $\text{var}(F_v)$ et joue le rôle d'arbre de référence pour la structuration du d-DNNF.

Lemme 3.6 (Lemme 5 de [17]). *Pour tout noeud v et toute paire (C, C') admise en v , le d-DNNF $\varphi_v(C, C')$ respecte $\text{vtree}(T, v)$. Le DAG construit est donc un d-DNNF structuré.*

Lemme 3.7 (Lemme 6 de [17]). *À la racine r de T , la seule paire admise est $(\emptyset, \emptyset)$, et $\varphi_r(\emptyset, \emptyset)$ est un d-DNNF structuré dont les modèles sont exactement les affectations de $\text{var}(F)$ satisfaisant F .*

Les trois lemmes ci-dessus assemblent un compilateur de F en d-DNNF structuré, sur lequel les requêtes polytimes étudiées en section 3.6 sont directement applicables. La taille du DAG produit fait l'objet de la section suivante.

3.5.3 Borne de taille (Lemme 7 et Théorème 1)

Lemme 3.8 (Lemme 7 de [17]). *Soit F une formule CNF de n variables et m clauses, et (T, δ) une branch decomposition de F de ps-width k . Le d-DNNF $\varphi_r(\emptyset, \emptyset)$ produit par la construction de la section 3.5.1 a une taille bornée par*

$$|\varphi_r(\emptyset, \emptyset)| \leq 7k^3(n + m).$$

La borne se justifie par décompte. Chaque triplet (C_1, C_2, C') de la section 3.3.4 ajoute au plus cinq noeuds et arêtes au DAG : un noeud AND, deux arêtes vers les fragments des enfants, une arête vers le noeud OR de la paire-parent, et éventuellement un nouveau noeud OR à allouer. Avec au plus k^3 triplets par noeud interne et $n + m - 1$ noeuds internes dans T , on obtient $5k^3(n + m - 1)$ noeuds et arêtes au plus. Chaque feuille ajoute au plus deux fragments mono-noeuds (littéraux ou constantes), soit $2(n + m)$ au total. La somme $5k^3(n + m - 1) + 2(n + m)$ se majore par $7k^3(n + m)$ pour $k \geq 1$.

Théorème 3.9 (Théorème 1 de [17]). *Toute formule CNF F de n variables, m clauses et de ps-width k admet une compilation en d-DNNF structuré de taille $O(k^3(n + m))$. L'algorithme de compilation associé s'exécute en temps $O(k^3 m(n + m))$.*

La borne en taille reprend le Lemme 3.8. Le temps incorpore le coût $O(m)$ par opération ensembliste, déjà présent dans la complexité du DP de Saether (Théorème 3.3). La validation empirique de cette borne sur les instances de test fait l'objet du chapitre 5.

Le d-DNNF produit appartient à la classe d-DNNF générale, plus large que celle des decision-DNNF (chapitre 2, section 2.5). Ce point n'a pas d'incidence sur l'applicabilité des requêtes polytimes étudiées au chapitre suivant.

3.6 Requêtes polytimes sur d-DNNF

La compilation BCMS (section 3.5) produit un d-DNNF qui sert de support à six requêtes calculables en temps polynomial, déjà nommées au chapitre 2, section 2.5.2 : consistance (CO), validité (VA), implication clausale (CE), test d'implicant (IM), comptage des modèles (CT, c'est-à-dire $\#SAT$) et énumération des modèles (ME). Le résultat fondateur est dû à Darwiche et Marquis [10], qui établissent que chacune de ces requêtes se réduit à un parcours du DAG dont la complexité est polynomiale en sa taille, dès que celui-ci satisfait certaines propriétés structurelles (sous-section 3.6.1).

3.6.1 Trois propriétés clés

Les trois propriétés clés du d-DNNF sont la *décomposabilité* (enfants d'un AND à portées variables disjointes), le *déterminisme* (enfants d'un OR à ensembles de modèles disjointes) et la *lissité* (enfants d'un OR à portées variables identiques). Les deux premières définissent la classe d-DNNF, déjà introduite au chapitre 2, section 2.5. La lissité, qui n'y figure pas, simplifie plusieurs des requêtes en garantissant que tous les sous-DAGs d'un OR parlent du même ensemble de variables.

Par le Lemme 3.5, le DAG construit en section 3.5 est décomposable et déterministe. Il n'est pas nécessairement lisse, mais peut être rendu tel à la demande par l'opération de lissage décrite en sous-section 3.6.4.

3.6.2 CT : model counting

Le comptage des modèles d'un d-DNNF φ se calcule par un parcours post-ordre du DAG. La fonction $ct(\varphi)$ se définit récursivement par

$$ct(\perp) = 0, \quad ct(\top) = 1, \quad ct(\ell) = 1 \text{ pour un littéral } \ell,$$

$$ct\left(\bigwedge_{i=1}^k \varphi_i\right) = \prod_{i=1}^k ct(\varphi_i), \quad ct\left(\bigvee_{i=1}^k \varphi_i\right) = \sum_{i=1}^k ct(\varphi_i).$$

La règle multiplicative aux noeuds AND est correcte par décomposabilité : les enfants ont des variables disjointes, leurs modèles se combinent indépendamment. La règle additive aux noeuds OR est correcte par déterminisme : les enfants ont des ensembles de modèles disjoints, leur somme ne double aucun compte. La mémorisation sur les identifiants des noeuds donne un parcours en temps $O(|D|)$, où $|D|$ désigne la taille du DAG (nombre de noeuds plus arêtes).

Appliqué à la racine $\varphi_r(\emptyset, \emptyset)$ du DAG produit par la compilation BCMS (section 3.5), ct livre directement $\#SAT(F)$: par le Lemme 3.7, les modèles de $\varphi_r(\emptyset, \emptyset)$ sur $\text{var}(F)$ sont précisément les affectations satisfaisantes de F .

3.6.3 CO, VA et ME

Trois requêtes se déduisent directement du comptage de sous-section 3.6.2.

La consistance (CO) revient à $\text{ct}(\varphi_r(\emptyset, \emptyset)) > 0$: le DAG est consistant si et seulement s'il a au moins un modèle. La décomposabilité seule suffit pour cette requête, car on s'intéresse à la positivité du compte et non à sa valeur.

La validité (VA) compare le compte au cardinal de l'espace d'affectations : φ_r est valide si et seulement si $\text{ct}(\varphi_r(\emptyset, \emptyset)) = 2^n$, où $n = |\text{var}(F)|$. Cette requête exige déterminisme et lissité pour que le compte soit exact sur l'espace total des affectations de $\text{var}(F)$.

L'énumération des modèles (ME) procède top-down par descente dans le DAG, guidée par le compte ct déjà calculé en sous-section 3.6.2. À chaque noeud AND, on collecte les choix de tous les enfants (variables disjointes par décomposabilité). À chaque noeud OR, on choisit un enfant à compte non nul. Aux feuilles littérales, on enregistre la valeur du littéral dans l'affectation en construction. La descente produit un modèle complet en $O(|D|)$.

Le même squelette donne l'énumération de *tous* les modèles : à chaque noeud OR, au lieu de choisir un enfant on itère sur tous les enfants à compte non nul, et à chaque noeud AND, le produit cartésien des sous-énumérations remplace la simple union des choix. Le délai entre deux modèles consécutifs reste $O(|D|)$, mais le nombre total de modèles peut être exponentiel. Sur un DAG non lisse, l'énumération peut produire le même modèle plusieurs fois, et l'opération de lissage de la section 3.6.4 y remédie.

3.6.4 Transformations internes : conditionnement et lissage

Deux transformations internes sur le d-DNNF préparent le DAG pour les requêtes CE (section 3.6.5) et IM (section 3.6.6).

Le *conditionnement* de φ par un littéral ℓ , noté $\varphi \mid \ell$, remplace chaque occurrence de ℓ par $\top$ et chaque occurrence de $\neg\ell$ par $\perp$ dans le DAG, puis propage les constantes par les règles habituelles ($\top \wedge X \equiv X$, $\perp \wedge X \equiv \perp$, $\top \vee X \equiv \top$, $\perp \vee X \equiv X$). Pour un terme $\gamma = \ell_1 \wedge \dots \wedge \ell_k$, on applique le conditionnement littéral par littéral. Le résultat $\varphi \mid \gamma$ a pour modèles les affectations $\tau : \text{var}(\varphi) \setminus \text{var}(\gamma) \rightarrow \{0, 1\}$ telles que $\tau \cup \{\ell_i \mapsto 1\}_{i=1}^k$ satisfait φ .

Le conditionnement préserve la décomposabilité et le déterminisme. Il ne préserve pas en général la lissité, car la propagation des constantes peut faire disparaître asymétriquement certaines variables des sous-DAGs d'un même OR, créant un OR à scopes hétérogènes.

Le *lissage* d'un d-DNNF rétablit la lissité quand elle a été perdue. Pour chaque noeud OR dont les enfants ont des scopes différents, on remplace chaque enfant ψ_i par $\psi_i \wedge \bigwedge_{x \in \text{missing}_i} (x \vee \neg x)$, où missing_i est l'ensemble des variables présentes dans au moins un autre enfant mais absentes de ψ_i . Les termes $(x \vee \neg x)$ sont identiquement vrais, donc les modèles du DAG sont inchangés et la lissité est restaurée.

Le lissage est nécessaire chaque fois qu'une requête compare le compte à une valeur absolue sur l'espace total des affectations (par exemple le test $\text{ct} = 2^{n-k}$ de IM, section 3.6.6). Pour CE en revanche (section 3.6.5), le test se réduit à $\text{ct} = 0$, qui est invariant par lissage : on peut omettre cette étape.

3.6.5 CE : clausal entailment

L'implication clausale teste si une clause $C = \ell_1 \vee \dots \vee \ell_k$ est conséquence logique de la formule F représentée par $\varphi_r(\emptyset, \emptyset)$. La réduction standard passe par la négation : $F \models C$ équivaut à dire qu'aucun modèle de F ne satisfait $\neg C = \neg\ell_1 \wedge \dots \wedge \neg\ell_k$. Cette condition s'écrit

$$F \models C \iff \text{ct}(\varphi_r(\emptyset, \emptyset) \mid \neg C) = 0.$$

On conditionne donc le DAG par le terme $\neg C$ (section 3.6.4) puis on calcule le compte du résultat (section 3.6.2). La clause C est conséquence de F si et seulement si ce compte est nul.

Le test étant fondé sur la nullité du compte, le lissage n'est pas requis : pour tout d-DNNF ψ , $\text{ct}(\psi) = 0$ équivaut à $\psi \equiv \perp$, indépendamment de la lissité. L'ensemble de la procédure (conditionnement par k littéraux et comptage) est en temps polynomial en $|D|$ et k .

3.6.6 IM : implicant check

Le test d'implicant prend en entrée un terme $\gamma = \ell_1 \wedge \dots \wedge \ell_k$ portant sur k variables distinctes, et détermine si toute affectation qui satisfait γ satisfait aussi F . La réduction passe par le conditionnement et un test d'égalité sur le compte :

$$\gamma \text{ implicant de } F \iff \text{ct}(\varphi') = 2^{n-k},$$

où $n = |\text{var}(F)|$ et φ' désigne le lissage de $\varphi_r(\emptyset, \emptyset) \mid \gamma$. Après conditionnement, le DAG résiduel encode $F \mid \gamma$ sur les $n - k$ variables restantes. Le terme γ est implicant si et seulement si toutes les 2^{n-k} affectations possibles de ces variables satisfont la formule, ce qui équivaut à compter exactement 2^{n-k} modèles.

À la différence de CE, le test compare le compte à une valeur absolue sur l'espace total, et non à zéro. Le compte doit donc être exact relativement à $\text{var}(F) \setminus \text{var}(\gamma)$, ce qui impose le lissage du DAG résiduel après conditionnement (section 3.6.4). L'enchaînement complet (conditionnement, lissage, comptage) est en temps polynomial en $|D|$ et k .

4

Implémentation

L'implémentation suit le pipeline théorique présenté au chapitre 3 en quatre phases successives : construction d'une branch decomposition (T, δ) par l'heuristique GreedyOrder, parcours bottom-up qui calcule les $\text{PS}'(F_v)$ (Procédure 1), parcours top-down qui calcule les $\text{PS}'(F_{\bar{v}})$ (Procédure 2), puis programmation dynamique de la Procédure 3 qui produit conjointement la valeur de MaxSAT, le compte $\# \text{SAT}$ et un DAG d-DNNF structuré au sens de [17]. Ce DAG sert ensuite de support aux six requêtes polytimes décrites en section 3.6.

Le code est écrit en C, sans dépendance hors de la bibliothèque standard. Un langage compilé donne une exécution plus rapide qu'un langage interprété, choix justifié par les nombreuses itérations de la programmation dynamique.

4.1 Représentation des PS-sets par bitsets 64-bit

Saether et al. [18] recommandent de représenter les PS-sets (section 3.2.2) comme des vecteurs booléens de longueur $|\text{cla}(F)|$, ce qui rend les opérations ensemblistes (test d'appartenance, union, intersection) en temps proportionnel à m . Notre implémentation prolonge ce choix par un packing 64 bits : chaque bitset est un tableau de `uint64_t` de taille $\lceil m/64 \rceil$, et toute opération ensembliste se réduit à un parcours mot par mot.

La formule F utilise la même représentation : pour chaque variable v , deux bitsets `mask_pos[v]` et `mask_neg[v]` encodent les clauses où v apparaît respectivement en littéral positif ou négatif.

4.2 Le Binary Trie : deduplication globale des PS-sets

Saether et al. [18] préconisent une structure de trie binaire pour gérer les PS-sets : un PS-set vu comme une suite de m bits définit un chemin de la racine à une feuille du trie, où chaque bit choisit l'enfant gauche (clause absente) ou l'enfant droit (clause présente). À la feuille correspondante, un identifiant entier unique est attribué la première fois que le PS-set est inséré, et retrouvé tel quel à toute insertion ultérieure. Insertion et recherche se font en $O(m)$, et le test d'égalité de deux PS-sets se ramène ensuite à une comparaison d'entiers en $O(1)$.

La structure concrète est un pool contigu de noeuds, alloué en bloc et redoublé chaque fois qu'il se remplit. La capacité initiale du pool vaut $2m^2$ dès que $m^2 > 10^5$, et 10^5 sinon ; cette valeur empirique couvre les instances de notre suite de tests sans réallocation. Chaque noeud du pool tient sur trois champs entiers : les deux index d'enfants et l'identifiant de PS-set (ou -1 si le noeud n'est pas terminal).

La déduplication des PS-sets calculés à un noeud v de la branch decomposition demande, à chaque insertion dans le PS-set local de v , de vérifier si l'identifiant a déjà été ajouté à v . Le coût naïf est $O(|\text{PS}'(F_v)|)$ par insertion. L'implémentation utilise un tableau `seen_array` indexé par identifiant de PS-set, qui stocke l'identifiant du noeud d'arbre en cours de traitement : le test `seen_array[ps_id] == node_id` suffit alors pour décider en $O(1)$ si le PS-set est déjà présent localement.

4.3 Procédures 1 et 2 : produits cartesiens et masques de coupe

Les Procédures 1 et 2 transcrivent directement les récurrences des sections 3.3.2 et 3.3.3. Le seul ressort propre à l'implémentation tient au masque de coupe et à sa gestion mémoire dans la Procédure 1.

Le masque de coupe est précalculé à la construction de l'arbre. Pour chaque noeud v , `delta_mask[v]` est un bitset sur les clauses qui matérialise $\text{cla}(F) \cap \delta(v)$, où $\delta(v)$ est la coupe associée au noeud (section 3.2.1). Il se calcule bottom-up par union des masques des enfants. La Procédure 2 utilise directement ce masque, puisque $\text{cla}(F_{\bar{v}}) = \text{cla}(F) \cap \delta(v)$. La Procédure 1 calcule à la volée `mask_Fv = delta_mask[v] ∩ all_clauses` à chaque appel récursif et le libère avant le retour, pour ne pas accumuler les bitsets temporaires sur la profondeur de la récursion.

Les cas de base aux feuilles et à la racine sont codés en dur via les masques `mask_pos` et `mask_neg`.

4.4 Procédure 3 : la boucle critique

La Procédure 3 mène dans une même triple boucle l'agrégat MaxSAT additif, le décompte #SAT multiplicatif et la construction du DAG d-DNNF (section 4.5). L'astuce de Bui-Xuan, Telle et Vatschelle (section 3.3.4) ramène les six indices d'une mise à jour de table à trois indices itérés explicitement : $(C_{c_1}, C_{c_2}, C'_v) \in \text{PS}'(F_{c_1}) \times \text{PS}'(F_{c_2}) \times \text{PS}'(F_{\bar{v}})$. Les trois indices restants C_v, C'_{c_1}, C'_{c_2} sont calculés à la volée par les identités déjà énoncées en section 3.3.4, puis insérés dans le trie pour récupérer leurs identifiants globaux.

Trois bitsets temporaires `temp_1`, `temp_2`, `temp_3` sont alloués une fois à l'entrée de la procédure et réutilisés à chaque triplet pour porter les calculs de C'_{c_1} , C'_{c_2} et C_v . Cette pré-allocation évite $\Theta(k^3)$ appels à `malloc` par noeud interne.

La boucle itère sur les tailles effectives des PS-sets, sans branche conditionnelle pour le cas linéaire. Sur un caterpillar (section 3.3.5), l'un des deux PS-sets enfants est de cardinal au plus deux, donc la triple boucle à un noeud interne parcourt $O(k^2)$ triplets au lieu de $O(k^3)$, et la borne du Théorème 3.4 est atteinte sans spécialisation du code.

4.5 Construction du DAG d-DNNF en parallèle du DP

La Procédure 3 construit, dans la même boucle que le DP, le DAG d-DNNF de l'équation (3.2). À chaque triplet (C_{c_1}, C_{c_2}, C'_v) qui met à jour `Tab_v(C_v, C'_v)`, un noeud AND est émis et raccroché au noeud OR de la cellule (C_v, C'_v) . Mener le DP et la compilation en même temps épargne ainsi une seconde passe sur les $\Theta(k^3)$ triplets par noeud interne. À la racine, $\varphi_r(\emptyset, \emptyset)$ est le DAG d-DNNF structuré calculant F (Lemme 3.5).

Le noeud OR d'une cellule est créé paresseusement au premier AND ajouté. Une cellule sans aucun AND valide reste vide et n'alloue aucun noeud, ce qui correspond aux cellules de compte #SAT nul. Tous les noeuds du DAG résident dans un pool unique alloué à l'entrée de la procédure et libéré en bloc à la fin, sans libération individuelle.

Le pool partage deux singletons pour les constantes $\top$ et $\perp$, ce qui ramène leur test d'égalité à une comparaison de pointeurs. La création d'un noeud AND applique alors les simplifications immédiates $\perp \wedge X \equiv \perp$ et $\top \wedge X \equiv X$ avant toute allocation, conformément à la propagation des constantes (section 3.5.1). La taille du DAG produit reste majorée par la borne $7k^3(n+m)$ du Lemme 3.8, et sera mesurée empiriquement au chapitre 5.

4.6 Heuristique GreedyOrder et modes alternatifs

L'implémentation de GreedyOrder (section 3.4) tient sur deux choix d'optimisation. La quantité qui guide la sélection, $Ldegree(v)$, compte les voisins de v dans $I(F)$ déjà placés dans l'ordre. À chaque itération, sa mise à jour ne porte que sur les voisins non choisis du dernier sommet sélectionné, ce qui évite un recalcul global. La sélection du meilleur candidat et cette mise à jour incrémentale coûtent chacune $O((n+m)^2)$ au total. À $Ldegree$ égal entre plusieurs candidats, le sommet de plus petit degré dans $I(F)$ est retenu (le moins de clauses pour une variable, le moins de variables pour une clause), conformément au pseudocode de [18]. Le caterpillar est assemblé au fil de la sélection : à chaque itération, la feuille du sommet retenu est greffée à la racine courante de l'arbre en construction.

Deux modes alternatifs sont disponibles via la ligne de commande et servent de baselines au chapitre 5. Le mode **random** fusionne les feuilles deux à deux dans un ordre aléatoire et produit en général une ps-width très élevée. Le mode **linear** construit un caterpillar déterministe selon l'ordre des variables DIMACS, chaque clause étant greffée juste après sa variable de plus grand indice. Ce dernier sert de référence pour mesurer l'apport propre de GreedyOrder par rapport à un ordre brut.

4.7 Requetes polytimes sur d-DNNF : choix d'implementation

Quatre des six requêtes se réduisent à un comptage des modèles ou à une descente top-down sur le DAG, selon les règles déjà énoncées en section 3.6.3. La fonction **count** mémorise les comptes par identifiant de noeud, ce qui permet aux quatre requêtes **consistency**, **validity**, **find_model** et **enumerate** de partager la même table lorsqu'elles sont enchaînées sur le même DAG.

Les deux dernières requêtes, **entails** et **is_implicant**, s'appuient sur la transformation **condition**, qui remplace les occurrences d'un littéral par $\top$ ou $\perp$ selon la valeur souhaitée et propage les constantes dans le DAG. Le code conserve volontairement les sous-DAGs $\top$ dans les noeuds OR, au lieu d'appliquer la simplification $\top \vee X \equiv \top$. Cette non-simplification préserve les variables apparaissant dans les sous-DAGs X , dont le lissage ultérieur a besoin pour insérer correctement les fragments $(v \vee \neg v)$ aux endroits manquants [10]. La transformation **smooth** matérialise ces fragments en exploitant une table de portées (les variables présentes dans chaque noeud, section 3.6.1), recalculée à chaque appel à partir de zéro.

entails et **is_implicant** composent **condition** avec **count**, et avec **smooth** pour **is_implicant**, selon les réductions des sections 3.6.5 et 3.6.6.

4.8 Robustesse : gestion memoire et erreurs

La gestion mémoire suit une discipline d'ownership stricte. Le pool du DAG (section 4.5) possède tous ses noeuds, le trie binaire (section 4.2) possède les siens, et le tableau de PS-sets stocké à chaque noeud v de l'arbre possède les bitsets qu'il contient. La libération suit un parcours post-ordre de l'arbre de décomposition : chaque appel libère d'abord les enfants, puis $PS'(F_v)$, $PS'(F_{\bar{v}})$ et le **delta_mask** du noeud courant. Les tables de DP ne touchent pas aux noeuds du

DAG, qui survivent jusqu'à la libération en bloc du pool. Les tables elles-mêmes sont stockées comme tableaux 1D indexés en row-major ($i \cdot \text{cols} + j$), ce qui évite la double indirection d'un tableau 2D classique et favorise la localité de cache dans les boucles internes de la Procédure 3.

Si une allocation interne échoue dans le trie binaire ou le pool du DAG, un drapeau est levé et toutes les opérations suivantes deviennent des no-ops renvoyant des valeurs neutres. L'appelant détecte ce drapeau et route vers une sortie d'erreur, ce qui évite un crash silencieux quand la mémoire vient à manquer.

5

Résultats expérimentaux

L'évaluation expérimentale poursuit trois buts. Premièrement, mesurer le coût réel de la programmation dynamique et de la compilation en d-DNNF sur des familles d'instances à ps-width contrôlée ou inconnue. Deuxièmement, valider empiriquement les bornes théoriques du chapitre 3 (taille du DAG, complexité, équivalence sémantique avec une formule CNF). Troisièmement, situer l'approche par rapport à un solveur de production (Z3) sur deux tâches distinctes : la résolution unique de #SAT/MaxSAT et l'amortissement de requêtes multiples sur un même DAG compilé.

5.1 Instances testées

Le benchmark évalue le pipeline DP+DAG sur six familles d'instances choisies pour leur structure : trois familles synthétiques tirées du papier de Saether et al. [18] (type 1, type 2, type 3), une famille `random` de k -SAT aléatoire qui sert de référence non structurée, et deux familles fournies par Loïc Bardi¹ qui représentent des circuits booléens réalistes (multiplication binaire et factorisation entière).

Type 1 : interval ordering libre. Le générateur tire $n + m$ intervalles sur la droite réelle. Une clause c contient une variable x si leurs intervalles se chevauchent, et la polarité du littéral est tirée uniformément. Le graphe d'incidence obtenu est un *interval bigraph*, structure pour laquelle Saether et al. (Corollaire 3) bornent la ps-width par $\min(m + 1, 2^t)$, avec t la taille maximale des clauses. La numérotation DIMACS des variables est ensuite permutée aléatoirement par défaut, ce qui efface la trace de l'interval ordering : sur la formule permutée, l'ordre DIMACS brut donne une mauvaise décomposition, et l'heuristique GreedyOrder (section 3.4) doit retrouver une décomposition proche de l'optimale à partir de la structure du graphe. Les variantes étiquetées `_ordered` préservent l'ordre des right endpoints d'intervalles et servent de baseline contrôle au mode `linear` en section 5.3.3.

Type 2 : interval ordering à taille fixe. Type 2 est la variante de type 1 dans laquelle la taille des clauses est forcée à exactement t (typiquement $t = 3$ ou $t = 4$), et où chaque intervalle de clause émet quatre clauses sur les mêmes variables avec des polarités tirées indépendamment. La borne de Saether reste $\min(m + 1, 2^t)$, mais l'instance est plus dense et beaucoup plus souvent UNSAT. La même politique de permutation et de variante `_ordered` s'applique.

1. Instances `tseytin` et `factorization` fournies par Loïc Bardi, Projet de Bachelor UNIGE 2024.

Type 3 : XOR circulaires. Type 3 construit la formule à partir de fonctions XOR à t variables, satisfaites quand le nombre de littéraux vrais est impair. Les n variables sont disposées en cercle aux positions $1, \dots, n$. Étant donné un pas $s \leq t$, le générateur place les XOR le long du cercle : la i -ème couvre les positions $[i \cdot s, \dots, i \cdot s + t - 1] \bmod n$, et l'on émet n/s XOR au total pour faire un tour complet. Quand $s < t$, deux XOR consécutives partagent $t - s$ variables, ce qui crée une chaîne de chevauchements le long du cercle. Chaque XOR est ensuite transcrit en 2^{t-1} clauses CNF (celles qui interdisent les affectations de parité paire des t variables concernées). Le graphe d'incidence obtenu est la bipartisation d'un graphe d'arcs circulaires, ce qui borne la ps-width par $\min(m^2 + 1, 4^t)$.

random : k -SAT aléatoire. La famille **random** génère des formules 3-SAT et 4-SAT selon le modèle uniforme classique : m clauses tirées indépendamment, chacune composée de k littéraux distincts à polarité aléatoire. Les variantes étiquetées **_difficile** fixent le ratio clauses sur variables près du seuil de satisfiabilité (autour de 4,27 pour $k = 3$), où les instances sont à la fois denses et incompressibles. Cette famille sert de référence non structurée : son graphe d'incidence n'a aucune propriété particulière, et la ps-width attendue est élevée.

tseytin : multiplication binaire $k \times k$. Les instances **tseytin** encodent en CNF la vérification d'une multiplication binaire $k \times k$, du circuit 3×3 au circuit 6×6 pour les tailles activées dans notre benchmark. Une variante **pure** restreint l'encodage aux portes essentielles. Le graphe d'incidence n'est pas un interval bigraph, et aucune borne théorique générique ne s'applique : on mesure la ps-width empiriquement.

factorization : factorisation 5x5. Les deux instances **factorization_5x5_464d** et sa variante simplifiée encodent la factorisation d'un produit binaire fixé sur cinq bits. Elles correspondent à des instances structurées non synthétiques (par opposition aux trois familles tirées du papier), et servent de charge réaliste pour le pipeline.

TABLE 5.1 – Couverture du benchmark par famille en mode **greedy**. La colonne « Échecs » agrège dépassements de la limite mémoire de 50 GiB par process, crashes et timeouts. Pages n , m et ps-width relevées sur les instances réussies. Le mode **linear** traite quelques instances supplémentaires (section 5.3.3).

Famille	# OK	# Échecs	n	m	ps-width
type1	11	3	20–200	24–213	12–21 000
type2	13	5	25–200	88–800	14–456
type3	19	1	30–5 000	60–10 000	6–64
random	11	11	7–30	20–85	32–16 400
tseytin	4	2	36–72	99–220	144–8 192
factorization	0	2			
Total	58	24			

Le benchmark teste au total 179 paires instance/mode d'arbre en passe A, dont 110 réussissent : 58 en mode **greedy**, 46 en mode **linear** et 6 en mode **random**. La table 5.1 montre que GreedyOrder couvre presque entièrement type 3 (19 sur 20), ramène la ps-width entre 6 et 64 y compris pour $n = 5 000$, et tient sur l'essentiel de type 1 et type 2. Les échecs se concentrent sur **random** (sept dépassements mémoire et quatre crashes parmi 22 instances tentées) et sur **factorization**, où **greedy** échoue sur les deux instances tandis que **linear** les traite avec une ps-width de 24 576.

Ces contrastes entre familles structurent les sections suivantes : **tseytin** et **random** concentrent les ps-width élevées et les pires temps de DP, alors que les trois familles synthétiques restent dans le régime polynomial visé par le papier.

Huit instances de très grande taille sont désactivées dans notre benchmark : quatre type 3 ($n \in \{50\,000, 100\,000\}$ ou $t = 5$ avec $n \in \{5\,000, 10\,000\}$) et quatre tseytin ($k \in \{7, 10\}$). Les activer aurait sensiblement allongé la durée totale du benchmark sans révéler de comportement qualitativement nouveau, leur ps-width attendue et leur temps de DP prolongeant les tendances déjà tracées par les tailles plus modestes de la même famille.

5.2 Méthodologie expérimentale

Trois axes structurent la méthodologie : la machine d'exécution (section 5.2.1), le protocole de mesure en trois passes complémentaires (section 5.2.2) et la validation des résultats par invariants théoriques (section 5.2.3). Les cas qui s'écartent du parcours nominal (échecs en exécution, instances désactivées) sont recensés en section 5.2.4.

5.2.1 Machine et environnement

Le benchmark s'exécute sur *racer*, une machine du laboratoire de **Arnaud Casteigts** équipée de deux processeurs AMD EPYC 9654 (192 cores physiques au total, 384 threads via SMT2, boost à environ 3,7 GHz) et de 755 GiB de mémoire vive, sous Ubuntu 24.04. Le solveur est compilé avec GCC 13.3 en -O3 et la version 4.16 de Z3 sert de référence externe.

Deux choix de configuration influencent les mesures. Chaque processus du solveur est borné à 50 GiB de mémoire résidente, ce qui transforme les explosions de ps-width en échecs propres plutôt qu'en saturation de la machine. Pour le chronométrage le plus isolé (section 5.2.2), les workers sont épinglés sur quatre cores n'appartenant pas au même groupe de partage de cache L3, ce qui réduit la contention sur les caches partagés. Les chiffres absolus présentés ensuite doivent être lus dans ce contexte, alors que les rapports (DP par rapport à Z3, **greedy** par rapport à **linear**) restent plus robustes au changement de machine.

5.2.2 Protocole de mesure en trois passes

Chaque instance traverse trois passes successives, chacune isolant un type de mesure différent.

La *passé A* est un large parcours parallèle : 64 jobs simultanés exécutent le solveur sur toutes les instances et tous les modes d'arbre, en parallèle d'un appel Z3 pour MaxSAT et d'un appel Z3 pour la satisfiabilité. Elle produit les mesures structurelles (ps-width, taille du DAG, #SAT, MaxSAT, temps par phase) sans isolation CPU stricte. C'est la passe qui permet de couvrir le plus grand nombre d'instances dans le budget temps imparti.

La *passé B* reprend chaque instance et la rejoue trois fois sur quatre workers épinglés (section 5.2.1). Les quatre temps obtenus sont agrégés en médiane, minimum, maximum et écart-type. C'est cette passe qui fournit les temps d'exécution chronométrés rapportés en section 5.3.

La *passé C* est plus spécifique : elle mesure le coût des requêtes décrites en section 3.6 sur un DAG déjà compilé, sans inclure le temps de compilation. Chaque requête (CO, VA, CT, ME, CE, IM) est exécutée cinq fois en mémoire dans un même processus, la première exécution étant écartée pour absorber l'effet de chauffe du cache. Cette passe alimente les résultats de la section 5.4 et l'analyse du break-even N.

Le timeout par instance vaut 1 800 s en passe A pour type 1, type 2 et random, et 3 600 s pour type 3, tseytin et factorization, qui peuvent demander plus de temps pour les plus grandes tailles. Les appels Z3 sont coupés à 600 s. La durée totale du benchmark est de 1 h 50 min.

5.2.3 Validation par invariants théoriques

Huit invariants sont vérifiés après chaque exécution. Ils croisent les sorties du DP, du DAG compilé et de Z3 pour détecter toute divergence sémantique ou tout dépassement de borne théorique. La table 5.2 les résume.

TABLE 5.2 – Invariants vérifiés sur les 164 exécutions réussies (toutes modes et passes confondues). « WARN » signale un dépassement *soft* d’une borne théorique générique (sans incidence sur la correction de l’algorithme). « Skip » compte les instances où l’invariant ne s’applique pas (par exemple Z3 en timeout pour les comparaisons croisées).

Sémantique de l’invariant	# OK	# WARN	# Skip
MaxSAT(DP) = MaxSAT(Z3)	164	0	69
#SAT(DP) = comptage sur le DAG	164	0	69
SAT(DP) cohérent avec SAT(Z3)	164	0	69
$ D \leq 7k^3(n + m)$ (Lemme 3.8)	161	0	72
ps-width $\leq$ borne théorique de la famille	51	33	149
temps par phase ≥ 0	164	0	69
CE(DAG) cohérente avec Z3	54	0	0
nombre de modèles énumérés = #SAT	15	0	39

Aucun invariant ne produit de FAIL hard sur l’ensemble du benchmark. Les 33 WARN portent uniquement sur l’invariant de ps-width par rapport à la borne théorique de la famille, et se répartissent en sept instances type 1, vingt instances type 2 et six instances type 3 où GreedyOrder rate la borne $\min(m + 1, 2^t)$ ou $\min(m^2 + 1, 4^t)$ du papier [18] sans pour autant invalider l’algorithme. L’analyse de ces WARN figure section 5.3.1.

5.2.4 Cas dégénérés

Sur les 82 paires (instance, mode **greedy**), 58 aboutissent et 24 échouent : 14 dépassements de la limite mémoire de 50 GiB, 8 segfaults sur des instances très grandes et 2 timeouts. Les dépassements mémoire correspondent à des ps-width effectives au-delà de la dizaine de milliers (familles **random_difficile**, certaines type 1 et type 2), où la construction des PS-sets explose. Les segfaults se concentrent sur les deux tailles extrêmes de type 3 ($n = 10\,000$) et sur les deux instances de factorization en mode **greedy**. Les timeouts sont la limite naturelle des plus grands circuits tseytin.

Les sections de résultats portent sur les 58 instances réussies en mode **greedy**. Les modes d’échec et leur lien avec la ps-width sont repris au chapitre 6.

5.3 Résultats : programmation dynamique et compilation

Cette section regroupe les mesures structurelles et chronométriques issues des passes A et B (section 5.2.2) sur les 58 instances réussies en mode **greedy** : ps-width effective par famille, temps de résolution comparés à Z3, impact du choix d'arbre, taille du DAG produit, et corrélation entre la ps-width mesurée et le nombre de conflits Z3.

5.3.1 ps-width observée et bornes théoriques

La ps-width est le paramètre central des bornes de complexité du chapitre 3 (Théorème 3.3 : $O(k^3 m(m+n))$ pour le DP, Lemme 3.8 : $O(k^3(n+m))$ pour le DAG). La table 5.3 récapitule les valeurs effectivement produites par GreedyOrder (section 3.4) sur les six familles.

TABLE 5.3 – ps-width observée en mode **greedy** sur les instances réussies. La colonne « Borne théorique » reprend les bornes établies par Saether et al. [18] (Corollaire 3) pour les familles concernées.

Famille	Médiane	Maximum	Borne théorique
type1	32	21 000	$\min(m+1, 2^t)$
type2	184	456	$\min(m+1, 2^t)$
type3	8	64	$\min(m^2+1, 4^t)$
random	415	16 400	(pas de borne)
tseytin	4 168	8 192	(pas de borne)

Type 3 reste très en dessous de sa borne : les instances $t = 3$ plafonnent à une ps-width de 8 (borne $4^3 = 64$), et le maximum observé de 64 provient des instances $t = 5$ (borne $4^5 = 1 024$). GreedyOrder y maintient la ps-width entre 6 et 64 y compris pour $n = 5 000$. Type 1 et type 2 dépassent en revanche la borne sur une part importante des instances : c'est l'origine de la majorité des 33 alertes `psw_within_family_bound` signalées en section 5.2.3 (7 pour type 1 et 20 pour type 2). Pour type 2, la borne nominale $2^t = 8$ avec $t = 3$ est presque toujours ratée, la médiane atteignant 184. L'écart vient principalement de l'heuristique gloutonne, qui ne reconnaît pas toujours l'interval ordering caché par la permutation aléatoire des identifiants (section 5.1). Les familles **random** et **tseytin**, sans borne théorique prédéfinie, montent à plusieurs milliers, ce qui annonce les temps de DP correspondants (section 5.3.2).

5.3.2 Temps DP par famille et comparaison à Z3

La table 5.4 donne les médianes de la passe B pour le DP en mode **greedy** et pour Z3 **Optimize** sur MaxSAT. La figure 5.1 reporte la même mesure par instance.

Z3 résout plus vite que le DP sur 55 des 58 instances comparées. Les trois cas où le DP l'emporte sont **type1_v20_c25** (0,6 ms contre 3,6 ms pour Z3) et les deux variantes permutée et ordonnée de **type3_n30_t3_s2** (gain inférieur à 10 %). Tous sont en deçà de 30 variables, dans la zone où l'*overhead* initial de Z3 pèse plus que le coût d'une passe DP. L'écart médian va d'un facteur 8 sur type 1 à 47 734 sur tseytin, où le DP s'écroule à cause des ps-width au-delà de 4 000 (table 5.3). Le pire cas absolu est **type1_v200_c250** (ps-width 21 000, 456 secondes pour le DP contre 5 ms pour Z3). Cette infériorité en single-shot était attendue : Z3 est un solveur industriel optimisé sur des décennies, alors que le DP poursuit une garantie de complexité paramétrée (Théorème 3.3) qui ne devient avantageuse que sur des classes structurées à ps-width réellement bornée. Type 3, où la ps-width reste basse, illustre cette zone favorable : le DP y prend des temps

TABLE 5.4 – Temps de résolution médian par famille en mode **greedy** (passe B) et facteur DP/Z3. Les colonnes « DP>Z3 » et « DP<Z3 » comptent les instances où DP est respectivement plus lent ou plus rapide que Z3.

Famille	DP (ms)	Z3 (ms)	DP/Z3	DP>Z3	DP<Z3
type1	11,9	1,4	8,3×	10	1
type2	125,7	3,3	38×	13	0
type3	143,1	4,5	32×	17	2
random	42,3	1,0	41×	11	0
tseytin	76 526,1	1,6	47 734×	4	0

comparables aux autres familles malgré $n = 5\,000$ variables, là où Z3 traite l'instance en quelques millisecondes par effort CDCL standard.

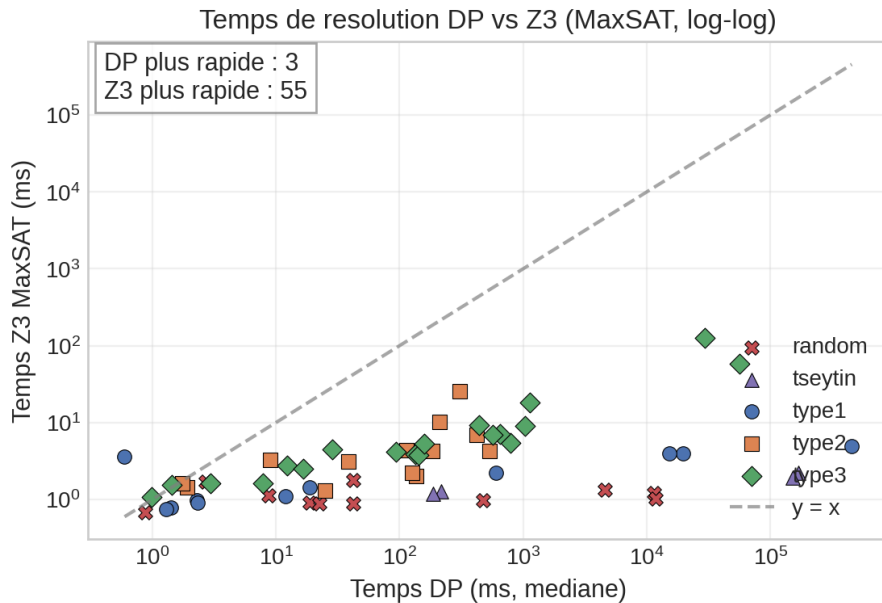


FIGURE 5.1 – Temps DP en mode **greedy** (abscisse) par rapport à Z3 **Optimize** sur MaxSAT (ordonnée), instance par instance. La diagonale $y = x$ sépare les instances où le DP est plus rapide (au-dessus) de celles où Z3 l'emporte (en-dessous). Trois instances se trouvent au-dessus de la diagonale, toutes de très petite taille ($n \leq 30$, $m \leq 60$).

5.3.3 Impact du mode de construction d'arbre

La figure 5.2 compare les ps-width obtenues par les deux modes **greedy** et **linear** sur les instances où les deux complètent. GreedyOrder produit systématiquement une décomposition meilleure ou équivalente, avec un ratio médian $\text{psw}_{\text{linear}}/\text{psw}_{\text{greedy}} = 3,1$ et un maximum à 6 144.

Le cas le plus extrême est **type3_n30_t3_s2** : GreedyOrder atteint $\text{psw} = 8$ (la borne 4^t théorique), alors que **linear** sur la même instance permutée monte à 49 152. L'ordre des variables DIMACS, arbitraire après la permutation aléatoire imposée par défaut (section 5.1), ne reflète plus la structure de la formule, et le mode **linear** le subit directement. Lorsque la permutation est désactivée (variantes **_ordered**), **linear** retrouve des ps-width proches de GreedyOrder, ce qui confirme que l'apport de l'heuristique est de récupérer un ordre adapté à partir du seul graphe d'incidence (section 4.6).

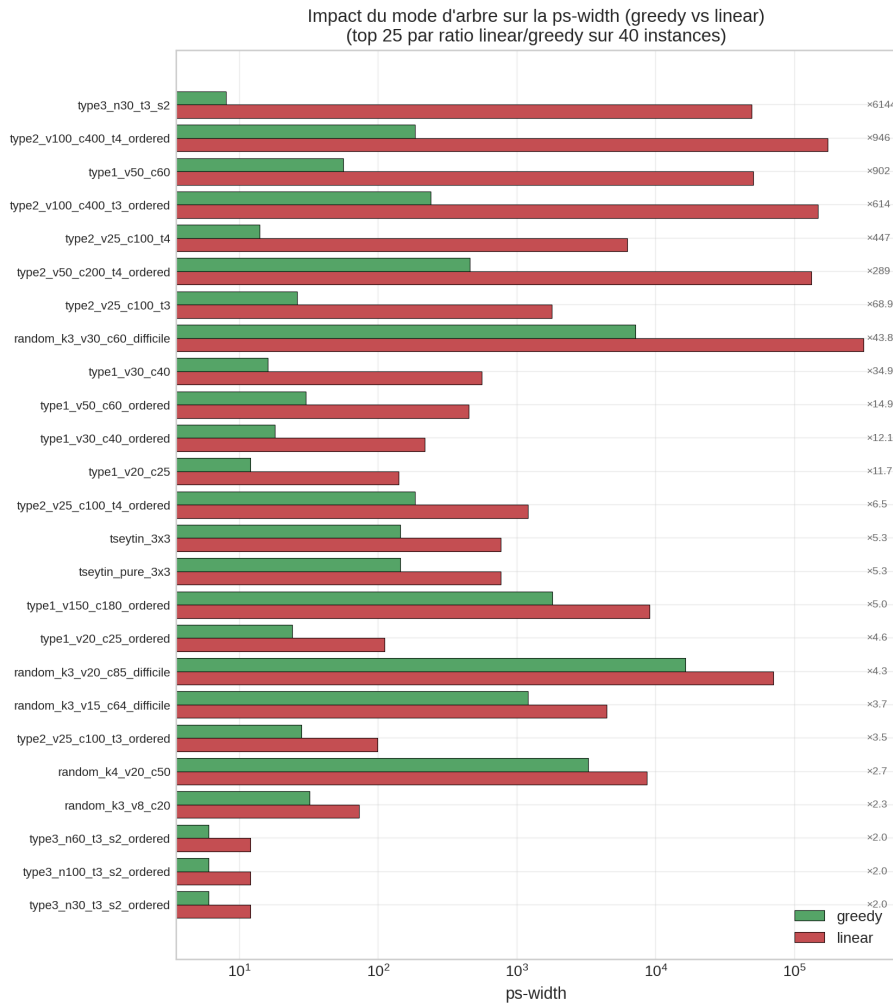


FIGURE 5.2 – ps-width obtenue par les modes **greedy** (vert) et **linear** (rouge) sur les 25 instances où l'écart est le plus marqué, parmi les 40 instances où les deux modes complètent. Les instances sont classées par ratio $\text{psw}_{\text{linear}}/\text{psw}_{\text{greedy}}$ (valeur indiquée à droite de chaque ligne).

5.3.4 Corrélation entre ps-width DP et conflits Z3

La figure 5.3 confronte la ps-width mesurée par le DP au nombre de conflits internes consommés par Z3 lors de sa résolution.

Les deux quantités ne sont pas alignées. Les instances tseytin, dont la ps-width grimpe entre 144 et 8 192, sont résolues par Z3 avec zéro ou deux conflits dans tous les cas observés : Z3 ne fait quasiment pas de backtrack, sa résolution est portée par la propagation unitaire seule. À l'inverse, type 2 montre une ps-width modérée (médiane 184) pour une médiane de huit conflits, le maximum observé restant en deçà de la centaine. Aucun cas du benchmark ne combine ps-width faible et nombre de conflits élevé, configuration qui aurait pu jouer en faveur du DP contre Z3. Ces observations confirment que la ps-width et l'effort CDCL mesurent des notions de difficulté différentes : la première est une propriété structurelle du graphe d'incidence (section 3.2.3), alors que la seconde reflète le comportement local d'un solveur d'inférence.

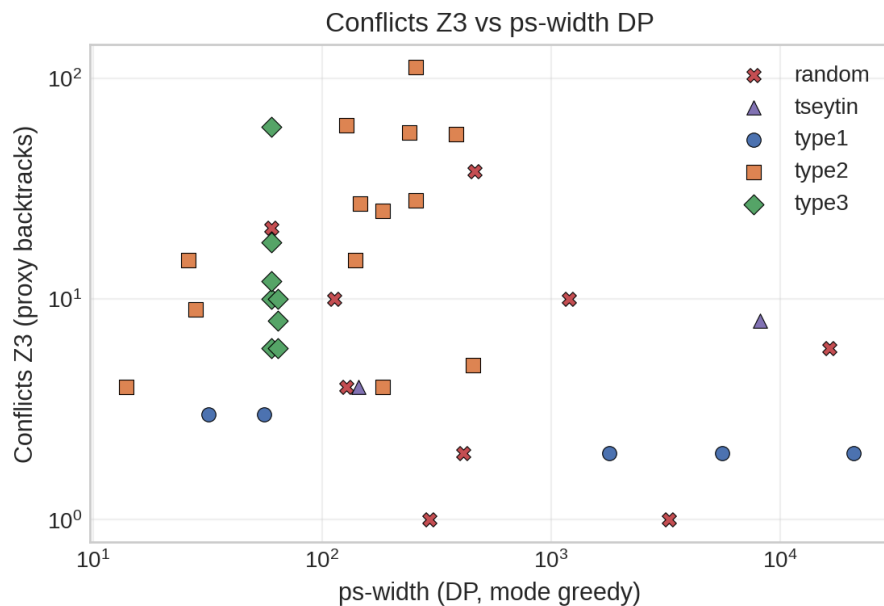


FIGURE 5.3 – Conflits Z3 par rapport à la ps-width effective (mode **greedy**), instance par instance. Chaque famille est représentée par une couleur distincte.

5.4 Résultats : requêtes sur DAG compilé

Une fois le DAG compilé, les requêtes polytimes de la section 3.6 s'exécutent par simple parcours, sans revenir au DP. La passe C mesure leur coût propre sur le DAG déjà construit (section 5.2.2), ce qui permet de quantifier le seuil à partir duquel ce coût de compilation est amorti par un usage répété.

5.4.1 Coût par requête

La table 5.5 donne le temps médian de chaque requête sur l'ensemble des DAG compilés, ainsi que son coût ramené à l'arête du DAG.

TABLE 5.5 – Coût médian des requêtes sur DAG compilé (passe C, médiane sur 5 répétitions après warm-up), sur les 39 instances à DAG non vide. Le coût par arête rapporte le temps à la taille du DAG. La ligne « tous les modèles » est mesurée sur les 15 instances dont l'ensemble des modèles reste énumérable (médiane de 47 modèles). Sur les autres, ce nombre dépasse les centaines de milliers et l'énumération complète n'est pas chronométrée.

Requête	Médiane (μ s)	Coût par arête (ns)
CO (consistance)	42,8	10,6
VA (validité)	44,0	10,5
CT (comptage #SAT)	42,6	10,5
CE (implication clausale)	295,6	66,9
IM (test d'implicant)	155,4	161,2
ME, un modèle	50,0	11,2
ME, tous les modèles	47,5	

Les requêtes CO, VA et CT coûtent entre 43 et 44 μ s, soit environ 10 ns par arête, ce qui reflète leur structure commune : un parcours unique du DAG en temps $O(|D|)$ (section 3.6.2). CE et IM sont plus chères (respectivement 296 et 155 μ s) car elles conditionnent d’abord le DAG, et IM le lisse ensuite, deux transformations qui le parcourent avant le comptage (section 4.7).

Les deux variantes de ME se distinguent par le traitement des noeuds OR. La recherche d’un modèle ne descend qu’une branche par OR et s’arrête au premier modèle trouvé (50 μ s). L’énumération parcourt au contraire toutes les branches pour produire l’ensemble des modèles. Sur les 15 instances où cet ensemble reste énumérable, elle prend 47,5 μ s, du même ordre que la recherche d’un seul modèle, car ces instances ont peu de modèles (médiane 47) et l’énumération ne précalcule pas la table des comptes nécessaire à la descente unique.

5.4.2 Comparaison CO vs Z3 SAT sur la même question

Parmi les six requêtes, seule la consistance (CO) pose à Z3 une question identique : la formule est-elle satisfaisable. Les autres (VA, CT, CE, IM, ME) répondent à des questions que Z3 ne traite pas nativement et ne donnent donc pas lieu à une comparaison directe. La figure 5.4 confronte uniquement CO sur DAG compilé à une décision SAT de Z3, sur une instance représentative de chaque famille.

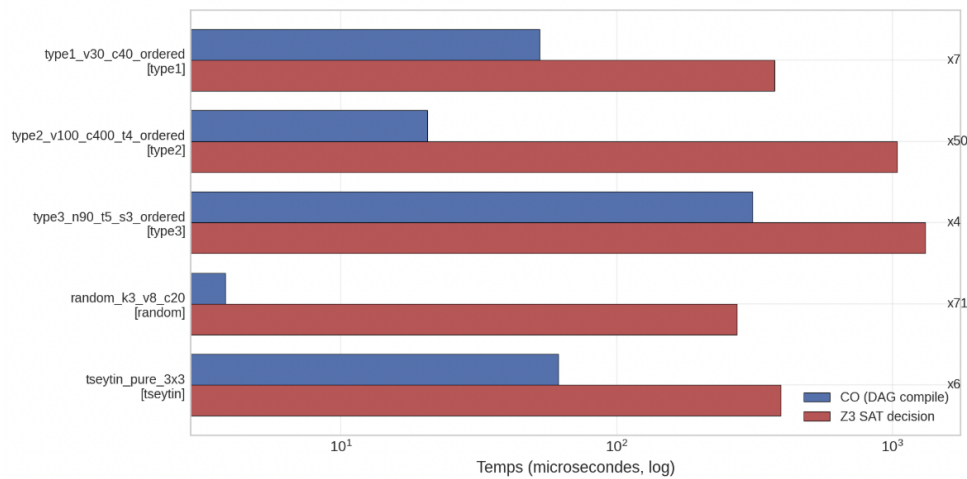


FIGURE 5.4 – Temps de la requête de consistance (CO) sur DAG compilé (bleu) face à une décision SAT de Z3 (rouge), sur une instance représentative par famille. Le facteur d’accélération est indiqué à droite. Échelle logarithmique en abscisse.

Sur ces instances, CO sur DAG va de 4 à 71 fois plus vite qu’un appel SAT de Z3. L’écart est le plus net sur **random_k3_v8_c20** ($\times 71$) et **type2_v100_c400_t4_ordered** ($\times 50$), et le plus mince sur **type3_n90_t5_s3_ordered** ($\times 4$). Cet avantage suppose que le DAG est déjà construit. Il faut aussi noter que Z3 est ici relancé à froid pour chaque instance, sans réutilisation d’état entre requêtes successives, et un solveur incrémental réduirait l’écart. Cette nuance, et la mise en balance avec le coût de compilation, sont reprises au chapitre 6.

5.4.3 Break-even N

La requête isolée est rapide, mais la compilation du DAG ne l’est pas (section 5.3). L’approche n’est rentable que si le DAG sert à plusieurs requêtes. Le seuil de rentabilité N^* est le nombre de requêtes au-delà duquel compiler le DAG puis poser N requêtes coûte moins cher que N appels

Z3 indépendants. La figure 5.5 trace les courbes de coût (panneau gauche) et la distribution de N^* sur l'ensemble des instances (panneau droit).

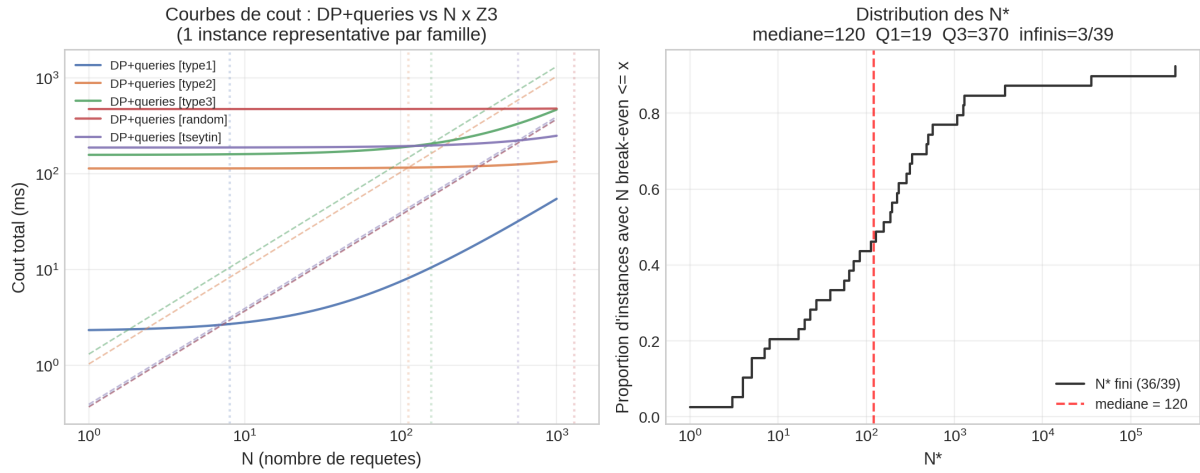


FIGURE 5.5 – À gauche : coût total du pipeline DP+requêtes en fonction du nombre N de requêtes (trait plein), comparé à N appels Z3 froids (pointillés), pour une instance représentative par famille. À droite : distribution cumulée du seuil N^* sur les instances où il est fini, médiane à 120.

La médiane de N^* vaut 120, avec un premier quartile à 19 et un troisième quartile à 370. Autrement dit, dès qu'une même formule fait l'objet d'environ 120 requêtes, le pipeline DP+DAG devient en moyenne plus économique que des appels Z3 répétés. Le seuil est fini sur 36 des 39 instances mesurées. Les trois exceptions, où aucun nombre raisonnable de requêtes n'amortit la compilation, sont les instances à ps-width élevée dont le DAG est le plus coûteux à produire. L'amortissement multi-requêtes est donc le cas d'usage où la compilation en d-DNNF prend tout son sens, par opposition à la résolution unique où Z3 domine (section 5.3.2).

6

Discussion

6.1 Du solveur au compilateur de connaissances

Pris comme solveur de #SAT et de MaxSAT à coup unique, l'algorithme de programmation dynamique de Saether et al. [18] ne rivalise pas avec un solveur de production. La section 5.3.2 chiffre l'écart : Z3 résout plus vite sur la quasi-totalité des instances comparées, et le facteur de ralentissement du DP atteint environ 47 734 sur les circuits tseytin. Cette infériorité était attendue. Z3 capitalise des décennies d'optimisation de la recherche CDCL, là où la programmation dynamique poursuit une garantie de complexité paramétrée par la ps-width, qui ne devient favorable que sur des familles réellement structurées.

L'intérêt de l'approche ne tient donc pas à sa vitesse de résolution. La même exécution qui calcule #SAT et MaxSAT construit un DAG d-DNNF structuré (section 3.5.1), et ce DAG est un objet réutilisable : une fois compilé, il répond à toute une famille de requêtes par simple parcours, sans relancer la programmation dynamique (section 5.4). La carte de compilation de Darwiche et Marquis [10] garantit que le d-DNNF rend polynomiales les six requêtes exploitées dans ce travail : consistance, validité, comptage des modèles, implication clausale, test d'implicant et énumération (section 3.6). La contribution de ce mémoire n'est ainsi pas un solveur plus rapide, mais un compilateur de connaissances.

La compilation a toutefois un coût, celui de la programmation dynamique elle-même (section 5.3), alors qu'une requête isolée sur le DAG est presque gratuite. L'approche n'est rentable que si le même DAG sert à plusieurs requêtes. La section 5.4.3 quantifie ce seuil : sur la médiane des instances mesurées, compiler le DAG puis poser $N^* = 120$ requêtes coûte autant que 120 appels Z3 indépendants, et l'avantage passe au compilateur au-delà. C'est dans cet usage répété, et non dans la résolution unique, que la compilation en d-DNNF prend sa valeur.

6.2 ps-width et effort CDCL : des difficultés orthogonales

La ps-width et l'effort de recherche d'un solveur CDCL mesurent deux difficultés sans rapport l'une avec l'autre. La section 5.3.4 le met en évidence : les instances tseytin atteignent une ps-width allant jusqu'à 8 192, là où Z3 les résout avec zéro à deux conflits. Une ps-width élevée n'entraîne donc pas une résolution coûteuse pour Z3, et rien dans nos mesures ne suggère la relation inverse. L'écart tient à la nature des deux quantités. La ps-width est une propriété structurelle du graphe d'incidence de la formule (section 3.2.3) et borne la largeur des états que la programmation dynamique maintient le long de la décomposition. Le nombre de conflits, lui,

mesure l'effort de recherche que Z3 consent au fil de sa résolution. Aucune des deux ne se déduit de l'autre.

Cette indépendance pèse directement sur la comparaison de la section 6.1. Le seul régime qui aurait avantage la programmation dynamique, une ps-width basse couplée à un grand nombre de conflits, n'apparaît dans aucune instance testée, tandis que les familles à ps-width élevée que le DP peine à traiter sont justement celles que Z3 résout sans effort. Aucun des deux paradigmes ne domine l'autre en général : chacun capture une forme de difficulté que l'autre ignore.

6.3 GreedyOrder, maillon faible et principal levier

La qualité des résultats du DP dépend entièrement de la branch decomposition fournie en entrée, et celle-ci provient de l'heuristique GreedyOrder, que Saether et al. [18] décrivent eux-mêmes comme « very simple ». Sur les familles structurées, cette simplicité se paie. Type 2 illustre le cas le plus net : sa borne théorique vaut $2^t = 8$ pour $t = 3$, alors que GreedyOrder y produit une ps-width de médiane 184 (section 5.3.1). Au total, 33 instances franchissent la borne de leur famille sans que l'algorithme en soit pour autant invalidé (section 5.2.3).

Ce dépassement est le point le plus coûteux de tout le pipeline. La complexité du DP croît comme le cube de la ps-width (Théorème 3.3), si bien que diviser la ps-width par deux divise le temps de résolution par environ huit. Aucun autre paramètre n'a un effet aussi marqué sur le coût. Or l'écart à la borne ne vient pas de la formule, mais de l'ordre dans lequel la décomposition est construite. La numérotation des variables est permutée aléatoirement par défaut, ce qui masque l'interval ordering sous-jacent, et l'heuristique ne le reconstitue pas toujours à partir du seul graphe d'incidence (section 5.3.3).

Deux voies pourraient resserrer cet écart. La première est exacte : pour les formules dont le graphe d'incidence est un *interval bigraph*, ce qui couvre les familles type 1 et type 2, le Théorème 6 de Saether et al. [18] fournit en temps polynomial une décomposition linéaire atteignant la borne $\min(m + 1, 2^t)$. Reconnaître ces formules et leur appliquer cette construction dédiée remplacerait l'heuristique par une décomposition optimale là où elle échoue le plus. La seconde est plus générale : les heuristiques de calcul de décompositions arborescentes, éprouvées de longue date pour minimiser la treewidth, pourraient être transposées à la recherche d'une branch decomposition de faible ps-width. Dans les deux cas, le constat reste le même. L'écart aux bornes du papier tient à l'heuristique de décomposition, non à l'algorithme de programmation dynamique, dont aucune exécution n'a produit de résultat incorrect (section 5.2.3).

6.4 Limites de l'évaluation et améliorations possibles

Plusieurs choix du protocole limitent la portée de la comparaison avec Z3, et tous flattent notre approche. Z3 est relancé à froid à chaque appel, sans réutilisation de son état interne d'une requête à la suivante. L'avantage mesuré de la consistance sur DAG (section 5.4.2) en ressort surévalué, puisqu'un usage incrémental de Z3 amortirait une partie de son coût de démarrage. Une comparaison à un Z3 incrémental, sur des séquences de requêtes proches, en donnerait une image plus juste. Plus fondamentalement, Z3 est un solveur SMT généraliste, ni un solveur de #SAT dédié, ni un compilateur de connaissances. Les solveurs de comptage spécialisés comme sharpSAT [7] et les compilateurs comme d4 [8] ou ganak offrirait une comparaison plus exigeante, que nous n'avons pas conduite, et les intégrer comme références situerait l'approche par rapport à l'état de l'art réel du domaine.

Deux autres réserves portent sur la portée des conclusions plutôt que sur la comparaison. Le benchmark ne retient que les instances menées à terme (section 5.2.4). Les plus difficiles dépassent la limite mémoire imposée et n'apparaissent pas dans les statistiques, qui décrivent

donc le comportement du DP sur les instances faisables et sous-représentent son pire cas. La borne de taille $7k^3(n + m)$ (Lemme 3.8) appelle une réserve du même ordre. Elle est respectée sur toutes les instances vérifiées (section 5.2.3), mais la taille réelle du DAG lui reste inférieure d'environ six ordres de grandeur en médiane. La borne garde sa valeur de garantie théorique et ne sert à rien pour dimensionner la mémoire d'une exécution concrète.

Ces réserves dessinent les prolongements naturels du côté de l'évaluation : intégrer un Z3 incrémental et des outils de #SAT spécialisés comme références, et repousser la limite mémoire pour mesurer le DP là où il échoue aujourd'hui. Avec les pistes algorithmiques de la section 6.3, elles tracent le programme d'une suite à ce travail, centrée sur la qualité de la compilation plutôt que sur la vitesse de résolution.

7

Conclusion

7.1 Bilan et réponse à la question de recherche

Ce mémoire visait deux objectifs complémentaires (chapitre 1), l'un d'implémentation, l'autre d'extension. Le premier est atteint : l'algorithme de programmation dynamique de Saether et al. [18] est implémenté de bout en bout en C, depuis la lecture des formules CNF jusqu'au calcul conjoint de #SAT et de MaxSAT, en passant par la construction d'une branch decomposition et les trois procédures du papier. Le second l'est aussi : la programmation dynamique est instrumentée à la manière de Bova, Capelli, Mengel et Slivovsky [17] pour produire, au cours de la même exécution, un DAG d-DNNF structuré, sur lequel les six requêtes que la carte de compilation de Darwiche et Marquis [10] rend polynomiales ont toutes été réalisées. Une évaluation empirique sur les familles synthétiques à ps-width contrôlée, les formules aléatoires et les circuits de multiplication accompagne ces deux volets (chapitre 5).

La cohérence de l'ensemble a été contrôlée par une batterie d'invariants théoriques (section 5.2.3). Sur toutes les instances vérifiées, la valeur de MaxSAT et le comptage des modèles coïncident avec ceux de Z3, le compte effectué sur le DAG redonne le #SAT de la programmation dynamique, et la taille du DAG respecte sa borne théorique, sans aucun échec. Théorie (chapitre 3) et implémentation (chapitre 4) concordent ainsi sur tout le benchmark.

L'introduction posait une question : dans quels régimes la décomposition structurelle apporte-t-elle un avantage ? La réponse est nuancée. Pour résoudre une formule une seule fois, elle n'en apporte aucun, et un solveur de production traite plus vite la quasi-totalité des instances (section 5.3.2). L'avantage se loge ailleurs, dans la réutilisation du DAG compilé : dès qu'une même formule fait l'objet d'environ $N^* = 120$ requêtes, le coût de compilation est amorti et le pipeline devient le plus économique (section 5.4.3). La décomposition structurelle ne livre donc pas un meilleur solveur, mais un compilateur de connaissances dont la valeur tient à l'usage répété (section 6.1).

7.2 Perspectives

La portée de ce résultat dépasse le cadre du benchmark. Un DAG compilé une fois sert de support à un raisonnement répété sur la même formule, qu'il s'agisse de compter ses modèles, d'en énumérer ou de tester des implications, sans jamais relancer la programmation dynamique (section 6.1). Les usages évoqués en introduction (chapitre 1), de l'inférence probabiliste exacte à la vérification quantitative, relèvent précisément de ce schéma, où une même théorie est interrogée un grand nombre de fois.

Le seuil de rentabilité mesuré (section 5.4.3) cerne le contexte où la compilation trouve son intérêt : l'interrogation répétée d'une formule, et non sa résolution ponctuelle. C'est là, plus que dans la course à la vitesse face aux solveurs de production, que la programmation dynamique sur branch decomposition et la compilation en d-DNNF prennent leur sens.

Bibliographie

- [1] S. A. Cook. The complexity of theorem-proving procedures. In *Proc. 3rd Annual ACM Symposium on Theory of Computing (STOC)*, pp. 151–158, 1971.
- [2] R. M. Karp. Reducibility among combinatorial problems. In R. E. Miller and J. W. Thatcher (eds.), *Complexity of Computer Computations*, pp. 85–103. Plenum Press, 1972.
- [3] L. G. Valiant. The complexity of enumeration and reliability problems. *SIAM Journal on Computing*, 8(3) :410–421, 1979.
- [4] M. W. Moskewicz, C. F. Madigan, Y. Zhao, L. Zhang, and S. Malik. Chaff : Engineering an efficient SAT solver. In *Proc. 38th Design Automation Conference (DAC)*, pp. 530–535, 2001.
- [5] N. Eén and N. Sörensson. An extensible SAT-solver. In *Proc. 6th International Conference on Theory and Applications of Satisfiability Testing (SAT)*, LNCS 2919, pp. 502–518. Springer, 2003.
- [6] J. P. Marques-Silva and K. A. Sakallah. GRASP : A search algorithm for propositional satisfiability. *IEEE Transactions on Computers*, 48(5) :506–521, 1999.
- [7] M. Thurley. sharpSAT – Counting models with advanced component caching and implicit BCP. In *Proc. 9th International Conference on Theory and Applications of Satisfiability Testing (SAT)*, LNCS 4121, pp. 424–429. Springer, 2006.
- [8] J.-M. Lagniez and P. Marquis. An improved decision-DNNF compiler. In *Proc. 26th International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 667–673, 2017.
- [9] L. de Moura and N. Bjørner. Z3 : An efficient SMT solver. In *Proc. 14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, LNCS 4963, pp. 337–340. Springer, 2008.
- [10] A. Darwiche and P. Marquis. A knowledge compilation map. *Journal of Artificial Intelligence Research (JAIR)*, 17 :229–264, 2002.
- [11] A. Darwiche. SDD : A new canonical representation of propositional knowledge bases. In *Proc. 22nd International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 819–826, 2011.
- [12] R. E. Bryant. Graph-based algorithms for Boolean function manipulation. *IEEE Transactions on Computers*, C-35(8) :677–691, 1986.
- [13] M. Samer and S. Szeider. Algorithms for propositional model counting. *Journal of Discrete Algorithms*, 8(1) :50–64, 2010.
- [14] F. Slivovsky and S. Szeider. A faster algorithm for propositional model counting parameterized by incidence treewidth. In *Proc. 23rd International Conference on Theory and Applications of Satisfiability Testing (SAT)*, LNCS 12178, pp. 267–276. Springer, 2020.
- [15] M. Vatshelle. *New Width Parameters of Graphs*. PhD thesis, University of Bergen, 2012.
- [16] B. Bergougnoux, É. Bonnet, and J. Duron. Mim-width is paraNP-complete. In *Proc. 52nd International Colloquium on Automata, Languages, and Programming (ICALP 2025)*, LIPIcs, 2025. DOI : 10.4230/LIPIcs.ICALP.2025.25.

-
- [17] S. Bova, F. Capelli, S. Mengel, and F. Slivovsky. On compiling CNFs into structured deterministic DNNFs. In *Proc. 18th International Conference on Theory and Applications of Satisfiability Testing (SAT)*, LNCS 9340, pp. 199–214. Springer, 2015.
 - [18] S. H. Saether, J. A. Telle, and M. Vatshelle. Solving #SAT and MaxSAT by dynamic programming. *Journal of Artificial Intelligence Research (JAIR)*, 54 :59–82, 2015.

Déclaration d'utilisation de l'IA

Je déclare avoir eu recours à des assistants d'intelligence artificielle selon les modalités suivantes :

- **Recherche et théorie** : l'IA a servi d'aide à la recherche bibliographique et à la consolidation de ma compréhension des concepts abordés dans ce travail.
- **Aide au développement** : l'IA a été utilisée comme partenaire de travail pour le débogage, l'optimisation, la suggestion de structures modulaires pour l'implémentation et la réalisation de figures soignées présentant les résultats. L'intégralité du code produit demeure néanmoins mon propre travail.
- **Rédaction et mise en forme** : l'IA m'a aidé pour la correction orthographique et grammaticale du texte et pour la reformulation de certains passages, afin d'en améliorer la clarté.